TRƯỜNG ĐẠI HỌC ĐÔNG Á

KHOA CNTT



BÁO CÁO MÔN HỌC
LẬP TRÌNH DI ĐỘNG 2

ĐỀ TÀI
XÂY DỰNG ỨNG DỤNG DI ĐỘNG GIÁM SÁT KHO ĐÔNG LẠNH

Sinh viên thực hiện: Vũ Huy Đạt
Lê Hà Thu Hằng
Lê Trung Kiên
Dương Xuân Huy
Nguyễn Đình Nhật Huy

Lớp: IT23M

Giảng viên hướng dẫn: Ths. Tạ Quốc Ý

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 3
1.1. Tổng quan về đề tài 3
1.2. Bối cảnh và lý do chọn đề tài 4
1.3. Mục tiêu của đề tài 5
1.4. Giải pháp thực hiện 6
1.5. Lý do lựa chọn nền tảng mô phỏng Wokwi 8
1.6. Phạm vi nghiên cứu của đề tài 9
1.7. Ý nghĩa thực tiễn của đề tài 10
CHƯƠNG 2: MỤC TIÊU VÀ PHẠM VI ĐỀ TÀI 10
2.1. Mục tiêu của đề tài 10
2.2. Các thông số vận hành của hệ thống 13
2.3. Phạm vi nghiên cứu và triển khai 14
2.4. Các giới hạn của đề tài 15
2.5. Kết quả mong đợi 16
CHƯƠNG 3: THÀNH PHẦN PHẦN CỨNG VÀ SƠ ĐỒ CHÂN 17
3.1. Tổng quan về phần cứng hệ thống 17
3.2. Bộ điều khiển trung tâm ESP32 17
3.3. Các cảm biến đầu vào 18
3.4. Thiết bị đầu vào mô phỏng trạng thái cửa kho 20
3.5. Các thiết bị đầu ra 21
3.6. Bảng phân công chân kết nối ESP32 22
3.7. Kết nối Wi-Fi và các thư viện sử dụng 23
3.8. Sơ đồ kết nối phần cứng 25
Chương 4: KIẾN TRÚC HỆ THỐNG 26
4.1. Tổng quan kiến trúc 26
4.2. Mô hình truyền dữ liệu 27
4.3. Chức năng của từng thành phần 28
4.4. Luồng xử lý cảnh báo và thông báo đẩy 31
4.5. Phạm vi triển khai và giới hạn nền tảng 31
CHƯƠNG 5: THIẾT KẾ PHẦN MỀM VÀ THUẬT TOÁN 32
5.1. Tổng quan thiết kế phần mềm 32
5.2. Thiết kế thuật toán chương trình ESP32 33
5.3. Bảng điều kiện xử lý và điều khiển 35

5.4. Thiết kế xử lý dữ liệu phía ứng dụng Web 36
5.5. Thiết kế Backend Node.js và Firebase Cloud Messaging 37
CHƯƠNG 6: GIAO THỨC MQTT VÀ ĐỊNH DẠNG DỮ LIỆU 38
6.1. Tổng quan giao thức MQTT 38
6.2. Mô hình kết nối MQTT 38
6.3. Kết quả mô phỏng và kịch bản kiểm thử 40
ẢNH THỰC TẾ BÀI LAB 41
CHƯƠNG 7: Kết luận 44

MỤC LỤC HÌNH ẢNH

Hình 3.8.1. Sơ đồ kết nối phần cứng hệ thống giám sát kho lạnh. 26
Hình 4.2.1. Kiến trúc tổng thể của hệ thống giám sát kho lạnh 29
Hình 6.1: Dữ liệu truyền trên MQTT 40
1. Điều khiển ánh sáng (Nháy Led, cảm biến ánh sáng) 41
2. Đọc Nhiệt độ và độ ẩm từ DHT11/ DHT22 ra log 42
3. Hiển thị nhiệt độ và độ ẩm từ DHT11 ra màn hình OLED 42
4. Đọc và hiển thị nhiệt độ, độ ẩm, khoảng cách (Cảm biến siêu âm), độ ẩm đất 44

MỤC LỤC BẢNG BIỂU

Bảng 2.1. Các thông số vận hành của hệ thống 15
Bảng 3.7.3. Các thư viện được sử dụng 26
Bảng 6.1. Thông số kết nối MQTT của hệ thống 40

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Tổng quan về đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ Internet of Things (IoT) đã mở ra nhiều hướng ứng dụng mới trong việc tự động hóa, giám sát và quản lý các hệ thống trong đời sống cũng như trong hoạt động sản xuất. Thông qua việc kết nối các thiết bị điện tử, cảm biến và hệ thống điều khiển với Internet, IoT cho phép thu thập, truyền tải, xử lý và hiển thị dữ liệu theo thời gian thực. Nhờ đó, con người có thể theo dõi tình trạng hoạt động của các thiết bị và môi trường từ xa mà không cần phải trực tiếp có mặt tại vị trí giám sát.

Một trong những lĩnh vực có nhu cầu ứng dụng công nghệ IoT cao là quản lý và giám sát kho lạnh, kho đông. Đây là những khu vực được sử dụng phổ biến trong việc bảo quản thực phẩm, thủy hải sản, vắc-xin, dược phẩm và nhiều loại hàng hóa yêu cầu điều kiện nhiệt độ đặc biệt. Để đảm bảo chất lượng hàng hóa trong suốt quá trình lưu trữ, kho lạnh cần duy trì các thông số môi trường, đặc biệt là nhiệt độ và độ ẩm, trong phạm vi cho phép.

Tuy nhiên, trong thực tế, việc giám sát kho lạnh bằng phương pháp thủ công vẫn còn tồn tại nhiều hạn chế. Người quản lý phải thường xuyên kiểm tra các thông số môi trường hoặc trực tiếp quan sát trạng thái hoạt động của kho. Điều này có thể dẫn đến tình trạng bỏ sót các sự cố, phát hiện bất thường không kịp thời hoặc không thể theo dõi liên tục khi không có mặt tại khu vực kho lạnh.

Xuất phát từ những vấn đề trên, đề tài "Xây dựng hệ thống IoT giám sát kho lạnh sử dụng ESP32 và giao thức MQTT" được thực hiện nhằm nghiên cứu, thiết kế và xây dựng một mô hình giám sát kho lạnh có khả năng thu thập dữ liệu môi trường, theo dõi trạng thái cửa kho, điều khiển thiết bị chiếu sáng và đưa ra cảnh báo khi phát hiện các điều kiện bất thường.

Hệ thống được triển khai dưới hình thức mô phỏng phần cứng ESP32 trên nền tảng Wokwi, kết hợp với giao thức MQTT để truyền dữ liệu đến ứng dụng web. Các thông tin thu thập được sẽ được hiển thị trực quan trên giao diện Dashboard, giúp người quản lý có thể theo dõi trạng thái kho lạnh theo thời gian thực, xem các thông số môi trường và kiểm tra lịch sử cảnh báo.

1.2. Bối cảnh và lý do chọn đề tài

Kho lạnh đóng vai trò quan trọng trong chuỗi cung ứng và bảo quản hàng hóa, đặc biệt đối với những sản phẩm nhạy cảm với sự thay đổi của nhiệt độ như thực phẩm tươi sống, thủy hải sản, vắc-xin và các loại dược phẩm. Việc duy trì môi trường bảo quản ổn định không chỉ giúp kéo dài thời gian sử dụng của sản phẩm mà còn góp phần đảm bảo chất lượng và an toàn trong quá trình lưu trữ.

Trong quá trình vận hành kho lạnh, có nhiều yếu tố có thể ảnh hưởng đến điều kiện bảo quản. Trong đó, hai vấn đề phổ biến cần được quan tâm là nhiệt độ vượt quá giới hạn cho phép và cửa kho bị mở trong thời gian dài.

Thứ nhất, khi nhiệt độ bên trong kho tăng cao hơn ngưỡng quy định, chất lượng hàng hóa có thể bị ảnh hưởng nghiêm trọng. Đối với thực phẩm, nhiệt độ không phù hợp có thể thúc đẩy quá trình phát triển của vi sinh vật, làm giảm chất lượng và rút ngắn thời gian bảo quản. Đối với vắc-xin và dược phẩm, sự thay đổi nhiệt độ ngoài phạm vi cho phép có thể làm giảm hiệu quả hoặc ảnh hưởng đến đặc tính của sản phẩm.

Thứ hai, cửa kho lạnh nếu không được đóng kín hoặc bị mở quá lâu sẽ khiến hơi lạnh thất thoát ra môi trường bên ngoài. Điều này làm nhiệt độ bên trong kho thay đổi, gây ảnh hưởng đến hàng hóa và khiến hệ thống làm lạnh phải hoạt động nhiều hơn để duy trì nhiệt độ ổn định. Hệ quả là mức tiêu thụ điện năng tăng lên, đồng thời làm giảm hiệu quả vận hành của kho lạnh.

Bên cạnh đó, việc thiếu hệ thống giám sát ánh sáng và điều khiển chiếu sáng tự động cũng có thể gây lãng phí năng lượng trong quá trình sử dụng kho. Nếu đèn vẫn hoạt động khi không cần thiết, chi phí điện năng sẽ tăng lên mà không mang lại hiệu quả sử dụng tương ứng.

Từ những vấn đề thực tế trên, việc xây dựng một hệ thống giám sát tự động có khả năng thu thập dữ liệu liên tục, phát hiện bất thường và gửi cảnh báo đến người quản lý là rất cần thiết.

Đề tài được lựa chọn nhằm vận dụng các kiến thức đã học về vi điều khiển, cảm biến, lập trình nhúng, truyền thông không dây, giao thức MQTT và phát triển ứng dụng web để xây dựng một mô hình IoT có tính ứng dụng thực tế. Đồng thời, đề tài cũng tạo điều kiện để tìm hiểu quy trình kết nối giữa thiết bị phần cứng, máy chủ trung gian và ứng dụng giám sát thông qua Internet.

1.3. Mục tiêu của đề tài

Đề tài hướng đến việc xây dựng một hệ thống IoT có khả năng giám sát và cảnh báo các điều kiện hoạt động của kho lạnh thông qua mô hình mô phỏng ESP32 và ứng dụng web.

Các mục tiêu cụ thể bao gồm:

- Giám sát nhiệt độ và độ ẩm: Sử dụng cảm biến DHT22 để thu thập dữ liệu nhiệt độ và độ ẩm trong môi trường kho lạnh, từ đó cung cấp thông tin phục vụ việc đánh giá điều kiện bảo quản.
- Theo dõi trạng thái cửa kho: Sử dụng nút nhấn để mô phỏng trạng thái đóng và mở cửa kho, giúp hệ thống nhận biết tình trạng cửa và phát hiện trường hợp cửa mở quá lâu.
- Giám sát cường độ ánh sáng: Sử dụng cảm biến quang trở LDR để nhận biết điều kiện ánh sáng bên trong kho, phục vụ việc điều khiển hệ thống chiếu sáng tự động.
- Điều khiển thiết bị chiếu sáng: Tự động bật hoặc tắt đèn kho dựa trên trạng thái ánh sáng hoặc điều kiện hoạt động được thiết lập.
- Cảnh báo bất thường: Sử dụng đèn LED và còi buzzer để đưa ra cảnh báo tại chỗ khi nhiệt độ vượt ngưỡng an toàn hoặc cửa kho mở trong thời gian quy định.
- Truyền dữ liệu qua MQTT: Thiết lập kết nối giữa ESP32 và máy chủ MQTT EMQX Cloud thông qua giao thức MQTT kết hợp TLS nhằm đảm bảo quá trình truyền dữ liệu được mã hóa.
- Xây dựng giao diện giám sát: Phát triển ứng dụng web bằng công nghệ Expo / React Native Web, cho phép hiển thị các thông số môi trường, trạng thái thiết bị và cảnh báo theo thời gian thực.
- Thống kê và quản lý dữ liệu: Xây dựng các thành phần Dashboard giúp người dùng theo dõi tình trạng kho lạnh, quan sát sự thay đổi của các thông số và xem danh sách cảnh báo phát sinh trong quá trình vận hành.

1.4. Giải pháp thực hiện

Để đạt được các mục tiêu đã đề ra, đề tài xây dựng hệ thống theo mô hình IoT gồm ba thành phần chính: thiết bị thu thập và xử lý dữ liệu, máy chủ trung gian truyền thông MQTT và ứng dụng web giám sát.

ESP32 được lựa chọn làm bộ điều khiển trung tâm của hệ thống nhờ khả năng xử lý tốt, tích hợp kết nối Wi-Fi và hỗ trợ nhiều giao tiếp với các thiết bị ngoại vi. Vi điều khiển này chịu trách nhiệm đọc dữ liệu từ cảm biến, xử lý các điều kiện kiểm tra và điều khiển các thiết bị đầu ra.

Các linh kiện được sử dụng trong mô hình bao gồm:

- ESP32: Đóng vai trò là bộ điều khiển trung tâm, thực hiện thu thập dữ liệu từ cảm biến, xử lý logic và truyền thông tin đến máy chủ MQTT.
- Cảm biến DHT22: Được sử dụng để đo nhiệt độ và độ ẩm môi trường bên trong kho lạnh.

- Nút nhấn (Push Button): Mô phỏng trạng thái đóng hoặc mở cửa kho, giúp kiểm tra thời gian cửa mở và phát hiện các trường hợp cửa chưa được đóng đúng quy định.
- Cảm biến quang trở (LDR): Đo cường độ ánh sáng tương đối trong kho, hỗ trợ xác định điều kiện chiếu sáng.
- Đèn LED: Mô phỏng hệ thống đèn chiếu sáng bên trong kho và đèn cảnh báo trạng thái bất thường.
- Còi Buzzer: Phát tín hiệu âm thanh khi hệ thống phát hiện các điều kiện nguy hiểm hoặc vượt ngưỡng cho phép.

Sau khi thu thập dữ liệu, ESP32 tiến hành xử lý và so sánh với các ngưỡng được cấu hình trước. Nếu phát hiện nhiệt độ vượt giới hạn an toàn hoặc cửa kho mở quá thời gian cho phép, hệ thống sẽ kích hoạt các thiết bị cảnh báo tại chỗ. Đồng thời, thông tin về trạng thái hoạt động sẽ được đóng gói và gửi lên máy chủ MQTT để ứng dụng web tiếp nhận.

1.4.2. Truyền dữ liệu thông qua giao thức MQTT

Để thực hiện việc truyền dữ liệu giữa ESP32 và ứng dụng web, đề tài sử dụng giao thức MQTT (Message Queuing Telemetry Transport). Đây là giao thức truyền thông theo mô hình Publish/Subscribe, được thiết kế để hoạt động hiệu quả trong các môi trường có băng thông hạn chế và phù hợp với các hệ thống IoT.

Trong mô hình này, EMQX Cloud Serverless được sử dụng làm MQTT Broker, đóng vai trò trung gian tiếp nhận và phân phối dữ liệu giữa các thiết bị.

Cụ thể, ESP32 hoạt động như một Publisher, có nhiệm vụ gửi dữ liệu cảm biến và trạng thái thiết bị lên các MQTT Topic đã được định nghĩa. Ứng dụng web hoạt động như một Subscriber, đăng ký theo dõi những Topic tương ứng để nhận dữ liệu do ESP32 gửi lên.

Việc sử dụng EMQX Cloud giúp hệ thống không cần tự xây dựng và duy trì một MQTT Broker riêng. Bên cạnh đó, kết nối MQTT được thiết lập thông qua TLS nhằm mã hóa dữ liệu trong quá trình truyền tải, góp phần nâng cao tính bảo mật và hạn chế nguy cơ thông tin bị đánh cắp khi giao tiếp qua Internet.

Thông qua cơ chế Publish/Subscribe, các thành phần trong hệ thống có thể trao đổi dữ liệu một cách linh hoạt, giảm sự phụ thuộc trực tiếp giữa thiết bị phần cứng và ứng dụng giám sát.

1.4.3. Xây dựng ứng dụng web giám sát

Ứng dụng giám sát được phát triển bằng Expo kết hợp React Native Web nhằm xây dựng giao diện có khả năng hoạt động trên trình duyệt web. Ứng dụng đóng vai trò là giao diện tương tác giữa hệ thống và người quản lý kho lạnh.

Thông qua việc kết nối đến MQTT Broker và đăng ký nhận dữ liệu từ các Topic tương ứng, ứng dụng có thể cập nhật thông tin từ ESP32 và hiển thị lên giao diện theo thời gian thực.

Các chức năng chính của ứng dụng bao gồm:

- Hiển thị nhiệt độ và độ ẩm hiện tại bên trong kho lạnh.
- Theo dõi trạng thái cửa kho, bao gồm trạng thái đóng và mở.
- Hiển thị thông tin về cường độ ánh sáng và trạng thái đèn chiếu sáng.
- Hiển thị trạng thái hoạt động của hệ thống cảnh báo LED và Buzzer.
- Cung cấp Dashboard trực quan để người dùng dễ dàng nhận biết tình trạng hoạt động của kho.
- Hiển thị danh sách các cảnh báo khi xảy ra những sự kiện bất thường.
- Thống kê và trực quan hóa dữ liệu nhằm hỗ trợ việc theo dõi sự thay đổi của các thông số môi trường.

Nhờ đó, người quản lý có thể theo dõi tình trạng kho lạnh từ xa thông qua thiết bị có kết nối Internet mà không cần trực tiếp kiểm tra các thông số tại khu vực kho.

1.5. Lý do lựa chọn nền tảng mô phỏng Wokwi

Trong quá trình nghiên cứu và triển khai các hệ thống IoT, việc thử nghiệm phần cứng thường đòi hỏi phải chuẩn bị nhiều linh kiện điện tử, thực hiện đấu nối mạch và kiểm tra hoạt động của từng thành phần. Điều này có thể phát sinh chi phí, mất nhiều thời gian và tiềm ẩn nguy cơ hư hỏng linh kiện do đấu nối sai hoặc lập trình chưa chính xác.

Để giải quyết những khó khăn trên, đề tài lựa chọn Wokwi làm nền tảng mô phỏng phần cứng ESP32.

Wokwi là nền tảng mô phỏng các mạch điện tử và vi điều khiển trực tuyến, hỗ trợ nhiều loại linh kiện phổ biến, trong đó có ESP32, cảm biến DHT22, LED, nút nhấn và các thiết bị ngoại vi khác. Nền tảng này cho phép người dùng thiết kế mạch, viết chương trình và kiểm tra hoạt động của hệ thống ngay trên trình duyệt mà không cần chuẩn bị phần cứng thực tế.

Một ưu điểm quan trọng của Wokwi là khả năng mô phỏng kết nối Wi-Fi thông qua mạng ảo Wokwi-GUEST. Nhờ đó, chương trình ESP32 có thể thực hiện các thao tác kết nối

Internet và giao tiếp với những dịch vụ bên ngoài, bao gồm máy chủ MQTT. Điều này cho phép đề tài kiểm thử quá trình truyền dữ liệu đến EMQX Cloud mà không cần sử dụng thiết bị ESP32 vật lý.

Ngoài ra, Wokwi cung cấp giao diện tương tác trực quan, cho phép người dùng thay đổi các giá trị cảm biến hoặc thao tác với nút nhấn ngay trong quá trình mô phỏng. Ví dụ, người thực hiện có thể thay đổi nhiệt độ và độ ẩm của cảm biến DHT22, nhấn nút để mô phỏng việc mở hoặc đóng cửa kho, đồng thời quan sát phản ứng của hệ thống thông qua LED và Buzzer.

Việc lựa chọn Wokwi mang lại một số lợi ích cụ thể như sau:

- Tiết kiệm chi phí: Không cần mua ESP32, cảm biến và các linh kiện điện tử để thực hiện các thử nghiệm ban đầu.
- Dễ dàng triển khai: Có thể thiết kế, lập trình và kiểm thử hệ thống trực tiếp trên trình duyệt web.
- Hạn chế rủi ro phần cứng: Giảm nguy cơ hư hỏng linh kiện do đấu nối sai hoặc cấp nguồn không phù hợp.
- Thuận tiện trong kiểm thử: Cho phép thay đổi các thông số đầu vào và quan sát kết quả hoạt động của hệ thống nhanh chóng.
- Hỗ trợ kết nối thực tế: Có thể mô phỏng kết nối Wi-Fi và giao tiếp với MQTT Broker bên ngoài, giúp kiểm thử quá trình truyền nhận dữ liệu.
- Dễ dàng chỉnh sửa và mở rộng: Có thể thay đổi sơ đồ mạch, bổ sung linh kiện và cập nhật chương trình mà không cần thay đổi phần cứng vật lý.

Như vậy, Wokwi là lựa chọn phù hợp để xây dựng và kiểm thử mô hình trong phạm vi đồ án, giúp tập trung nghiên cứu vào kiến trúc hệ thống, xử lý dữ liệu cảm biến và quá trình giao tiếp giữa thiết bị IoT với ứng dụng web.

1.6. Phạm vi nghiên cứu của đề tài

Trong phạm vi đồ án, hệ thống tập trung nghiên cứu và triển khai các chức năng giám sát cơ bản của một kho lạnh thông qua mô hình IoT mô phỏng.

Các nội dung được giới hạn như sau:

- Sử dụng ESP32 làm vi điều khiển trung tâm và triển khai trên nền tảng mô phỏng Wokwi.
- Sử dụng cảm biến DHT22 để thu thập dữ liệu nhiệt độ và độ ẩm.
- Sử dụng nút nhấn để mô phỏng trạng thái cửa kho.
- Sử dụng cảm biến LDR để theo dõi cường độ ánh sáng.

- Điều khiển LED và Buzzer dựa trên các điều kiện được lập trình.
- Sử dụng EMQX Cloud Serverless làm máy chủ MQTT trung gian.
- Sử dụng giao thức MQTT kết hợp TLS để truyền dữ liệu giữa ESP32 và ứng dụng web.
- Xây dựng giao diện giám sát bằng Expo / React Native Web.
- Hiển thị dữ liệu cảm biến, trạng thái thiết bị, Dashboard và danh sách cảnh báo.

Đề tài chủ yếu tập trung vào chức năng giám sát, xử lý và cảnh báo. Hệ thống chưa triển khai điều khiển trực tiếp máy nén lạnh hoặc các thiết bị làm lạnh công nghiệp. Các thông số ngưỡng nhiệt độ, độ ẩm và thời gian mở cửa được thiết lập nhằm phục vụ mục đích mô phỏng và kiểm thử, không thay thế các tiêu chuẩn bảo quản chuyên biệt của từng loại hàng hóa.

1.7. Ý nghĩa thực tiễn của đề tài

Đề tài có ý nghĩa trong việc nghiên cứu và ứng dụng công nghệ IoT vào bài toán giám sát môi trường bảo quản hàng hóa. Thông qua việc kết hợp vi điều khiển ESP32, cảm biến, giao thức MQTT và ứng dụng web, hệ thống minh họa được quy trình hoạt động cơ bản của một giải pháp giám sát từ xa.

Về mặt học thuật, đề tài giúp củng cố kiến thức về lập trình vi điều khiển, giao tiếp với cảm biến, xử lý tín hiệu đầu vào, điều khiển thiết bị ngoại vi và truyền dữ liệu qua mạng Internet. Đồng thời, người thực hiện có cơ hội tìm hiểu cách triển khai mô hình Publish/Subscribe của MQTT, sử dụng dịch vụ MQTT Broker trên nền tảng đám mây và xây dựng ứng dụng web có khả năng cập nhật dữ liệu theo thời gian thực.

Về mặt thực tiễn, mô hình có thể được phát triển thành một hệ thống giám sát kho lạnh hoàn chỉnh hơn trong tương lai. Khi thay thế phần cứng mô phỏng bằng thiết bị thực tế, bổ sung các cảm biến chuyên dụng và tích hợp thêm chức năng lưu trữ dữ liệu, hệ thống có thể hỗ trợ người quản lý phát hiện sớm các sự cố, hạn chế rủi ro hư hỏng hàng hóa và nâng cao hiệu quả quản lý kho.

CHƯƠNG 2: MỤC TIÊU VÀ PHẠM VI ĐỀ TÀI

2.1. Mục tiêu của đề tài

Mục tiêu chính của đề tài là thiết kế, xây dựng và kiểm chứng một hệ thống IoT giám sát, cảnh báo kho lạnh thông qua mô hình mô phỏng phần cứng ESP32 trên nền tảng Wokwi. Hệ thống được xây dựng nhằm tự động thu thập các thông số môi trường, theo dõi trạng thái đóng/mở cửa kho, xử lý dữ liệu và phát hiện những tình huống có nguy cơ ảnh hưởng đến điều kiện bảo quản hàng hóa.

Bên cạnh chức năng giám sát tại chỗ, hệ thống còn hướng đến khả năng truyền dữ liệu lên nền tảng điện toán đám mây thông qua giao thức MQTT, từ đó cho phép người dùng theo dõi tình trạng kho lạnh trên ứng dụng web theo thời gian thực. Khi phát hiện các điều kiện bất thường, hệ thống sẽ kích hoạt các thiết bị cảnh báo tại chỗ, đồng thời gửi thông tin cảnh báo đến ứng dụng để người quản lý có thể nhanh chóng nắm bắt và đưa ra biện pháp xử lý phù hợp.

Cụ thể, đề tài hướng đến việc hoàn thiện một chu trình hoạt động gồm các giai đoạn: Thu thập dữ liệu → Xử lý dữ liệu → Phát hiện bất thường → Cảnh báo tại chỗ → Truyền dữ liệu lên Cloud → Hiển thị và giám sát trên Web.

Hệ thống sử dụng cảm biến DHT22 để đo nhiệt độ và độ ẩm bên trong kho lạnh. Đây là hai thông số quan trọng ảnh hưởng trực tiếp đến điều kiện bảo quản hàng hóa. Dữ liệu sau khi được cảm biến thu thập sẽ được gửi đến ESP32 để xử lý và phục vụ quá trình giám sát.

Ngoài ra, hệ thống sử dụng cảm biến quang trở LDR để đo cường độ ánh sáng tương đối trong kho. Thông tin từ cảm biến này được sử dụng để xác định điều kiện ánh sáng, hỗ trợ chức năng điều khiển đèn chiếu sáng tự động khi có sự thay đổi trạng thái cửa kho.

Thông qua việc kết hợp các cảm biến, hệ thống có thể thu thập nhiều loại dữ liệu môi trường và trạng thái hoạt động khác nhau, tạo cơ sở cho việc giám sát và phát hiện bất thường.

Đề tài sử dụng nút nhấn để mô phỏng trạng thái đóng và mở cửa kho lạnh. Khi người dùng thực hiện thao tác nhấn hoặc nhả nút, ESP32 sẽ nhận biết sự thay đổi trạng thái và cập nhật thông tin tương ứng.

Đặc biệt, hệ thống không chỉ xác định cửa đang mở hay đóng mà còn thực hiện tính toán thời gian cửa mở liên tục. Khi cửa được mở, bộ điều khiển bắt đầu ghi nhận thời điểm mở cửa. Nếu cửa tiếp tục ở trạng thái mở, hệ thống sẽ tính toán khoảng thời gian đã trôi qua.

Khi thời gian mở cửa vượt quá giới hạn được thiết lập, hệ thống sẽ xác định đây là tình huống bất thường và kích hoạt cơ chế cảnh báo. Ngược lại, khi cửa được đóng lại, bộ đếm thời gian mở cửa sẽ được kết thúc và đưa về trạng thái phù hợp.

Chức năng này giúp mô phỏng việc kiểm soát tình trạng thất thoát hơi lạnh do cửa kho bị mở quá lâu.

Hệ thống được thiết kế với khả năng điều khiển các thiết bị đầu ra thông qua ESP32 nhằm hỗ trợ quá trình vận hành kho lạnh.

Đối với hệ thống chiếu sáng, đèn kho sẽ được bật khi phát hiện cửa kho mở, mô phỏng tình huống người dùng đi vào bên trong kho. Khi cửa được đóng lại, hệ thống sẽ điều chỉnh trạng thái đèn theo logic điều khiển đã được thiết lập.

Đối với chức năng cảnh báo, hệ thống sử dụng đèn LED và còi Buzzer để phát tín hiệu khi xuất hiện các điều kiện bất thường. Cụ thể, cảnh báo sẽ được kích hoạt trong hai trường hợp:

- Nhiệt độ trong kho vượt quá ngưỡng an toàn đã được cấu hình.
- Thời gian cửa kho mở liên tục vượt quá thời gian cho phép.

Việc kết hợp cảnh báo bằng ánh sáng và âm thanh giúp người dùng có thể nhận biết sự cố ngay tại khu vực kho, từ đó chủ động kiểm tra và xử lý kịp thời.

Một trong những mục tiêu quan trọng của đề tài là xây dựng cơ chế truyền dữ liệu từ ESP32 đến máy chủ đám mây nhằm phục vụ việc giám sát từ xa.

Hệ thống sử dụng giao thức MQTT theo mô hình Publish/Subscribe, kết hợp với dịch vụ EMQX Cloud Serverless làm MQTT Broker trung gian. ESP32 đóng vai trò Publisher, thực hiện gửi các thông tin về nhiệt độ, độ ẩm, trạng thái cửa, thời gian mở cửa và tình trạng cảnh báo lên các Topic đã được định nghĩa.

Dữ liệu được đóng gói dưới định dạng JSON để thuận tiện cho việc truyền tải, phân tích và xử lý tại ứng dụng web.

Để tăng cường tính bảo mật trong quá trình giao tiếp, hệ thống sử dụng kết nối MQTT/TLS thông qua cổng 8883. Cơ chế này giúp mã hóa dữ liệu truyền giữa thiết bị và MQTT Broker, hạn chế nguy cơ thông tin bị truy cập trái phép trong quá trình trao đổi qua Internet.

Đề tài hướng đến việc xây dựng một giao diện web có khả năng tiếp nhận và hiển thị dữ liệu được gửi từ ESP32 thông qua EMQX Cloud.

Ứng dụng được phát triển bằng Expo kết hợp React Native Web, cung cấp giao diện trực quan giúp người dùng theo dõi tình trạng hoạt động của kho lạnh mà không cần truy cập trực tiếp vào thiết bị mô phỏng.

Các thông tin được hiển thị trên Dashboard bao gồm:

- Nhiệt độ hiện tại của kho lạnh.
- Độ ẩm môi trường bên trong kho.
- Trạng thái cửa kho (đang mở hoặc đã đóng).
- Thời gian cửa kho mở liên tục.
- Lịch sử thay đổi các thông số môi trường.
- Danh sách các cảnh báo được phát sinh trong quá trình vận hành.

Thông qua giao diện này, người dùng có thể nhanh chóng đánh giá tình trạng kho lạnh, nhận biết các sự cố và theo dõi những thay đổi của môi trường trong quá trình hoạt động.

2.2. Các thông số vận hành của hệ thống

Để đảm bảo hệ thống hoạt động thống nhất và có cơ sở kiểm thử, đề tài thiết lập một số thông số vận hành cụ thể. Các thông số này được sử dụng làm điều kiện đầu vào cho quá trình xử lý dữ liệu và xác định trạng thái cảnh báo.

STT	Thông số	Giá trị thiết lập	Ý nghĩa
1	Ngưỡng nhiệt độ an toàn	-15,0 °C	Giới hạn nhiệt độ kho đông
2	Thời gian cửa mở tối đa	180 giây	Giới hạn thời gian mở cửa liên tục
3	Chu kỳ gửi dữ liệu	5 giây	Khoảng thời gian giữa hai lần gửi telemetry

Bảng 2.1. Các thông số vận hành của hệ thống

Nhiệt độ được xem là một trong những yếu tố quan trọng nhất đối với hoạt động bảo quản trong kho đông. Trong phạm vi đề tài, ngưỡng nhiệt độ an toàn được thiết lập ở mức -15,0 °C.

ESP32 sẽ liên tục tiếp nhận dữ liệu từ cảm biến DHT22 và thực hiện so sánh với ngưỡng nhiệt độ đã cấu hình. Khi nhiệt độ đo được vượt quá giới hạn này, hệ thống sẽ xác định trạng thái nhiệt độ không an toàn và kích hoạt đèn LED cùng còi Buzzer để cảnh báo.

Đồng thời, thông tin nhiệt độ và trạng thái cảnh báo sẽ được gửi lên MQTT Broker để ứng dụng web cập nhật và hiển thị cho người quản lý.

Việc cửa kho mở quá lâu có thể khiến hơi lạnh thất thoát và làm ảnh hưởng đến nhiệt độ bên trong kho. Vì vậy, hệ thống thiết lập thời gian cửa mở liên tục tối đa là 180 giây (3 phút).

Khi phát hiện cửa kho được mở, ESP32 bắt đầu tính thời gian. Nếu cửa vẫn tiếp tục mở và thời gian vượt quá 180 giây, hệ thống sẽ kích hoạt cảnh báo bằng LED và Buzzer.

Khi cửa được đóng lại trước hoặc sau thời điểm cảnh báo, hệ thống sẽ cập nhật trạng thái cửa và xử lý bộ đếm thời gian theo chương trình điều khiển.

Để đảm bảo dữ liệu trên Dashboard được cập nhật thường xuyên, hệ thống thiết lập chu kỳ gửi dữ liệu lên MQTT Broker là 5 giây/lần.

Theo đó, cứ sau mỗi khoảng thời gian 5 giây, ESP32 sẽ tổng hợp các thông tin cần thiết, bao gồm nhiệt độ, độ ẩm, trạng thái cửa, thời gian mở cửa và trạng thái cảnh báo, sau đó đóng gói dữ liệu dưới dạng JSON và gửi lên máy chủ EMQX Cloud.

Chu kỳ này giúp ứng dụng web có thể cập nhật các thông số gần với thời gian thực, đồng thời hạn chế việc gửi dữ liệu quá thường xuyên gây lãng phí tài nguyên truyền thông.

2.3. Phạm vi nghiên cứu và triển khai

Toàn bộ phần cứng của hệ thống được xây dựng và mô phỏng trên nền tảng Wokwi. Các linh kiện như ESP32, cảm biến DHT22, cảm biến LDR, nút nhấn, đèn LED và còi Buzzer được sử dụng để mô phỏng hoạt động của một kho lạnh.

Đề tài không thực hiện mua sắm linh kiện, đấu nối mạch điện hoặc triển khai hệ thống trên thiết bị phần cứng thực tế. Mọi hoạt động kiểm thử cảm biến, điều khiển thiết bị và xử lý tín hiệu đều được thực hiện trong môi trường mô phỏng.

Hệ thống sử dụng EMQX Cloud Serverless làm nền tảng MQTT Broker để tiếp nhận và phân phối dữ liệu từ ESP32 đến ứng dụng web.

Kết nối giữa ESP32 và EMQX Cloud được thực hiện thông qua giao thức MQTT/TLS, sử dụng cổng 8883. Các thông tin truyền tải được tổ chức dưới dạng JSON nhằm đảm bảo tính thống nhất trong quá trình trao đổi dữ liệu.

Trong phạm vi đề tài, trọng tâm của phần truyền thông là quá trình gửi và nhận dữ liệu thông qua MQTT. Việc lưu trữ lâu dài, phân tích dữ liệu lớn hoặc tích hợp với các hệ thống quản lý kho lạnh chuyên nghiệp chưa thuộc phạm vi triển khai chính.

Ứng dụng web được xây dựng bằng Expo / React Native Web và kết nối trực tiếp đến EMQX Cloud để đăng ký nhận dữ liệu từ các MQTT Topic.

Ứng dụng có nhiệm vụ hiển thị dữ liệu môi trường, trạng thái cửa, lịch sử hoạt động và danh sách cảnh báo. Các thông tin được cập nhật dựa trên dữ liệu thực tế được truyền từ chương trình ESP32 đang chạy trên Wokwi.

Do đó, phiên bản web là thành phần trực tiếp sử dụng dữ liệu được truyền qua MQTT Broker, phục vụ việc kiểm chứng khả năng giao tiếp giữa thiết bị mô phỏng và ứng dụng giám sát.

Bên cạnh phiên bản web, dự án có hỗ trợ giao diện ứng dụng dành cho nền tảng Android và iOS thông qua hệ sinh thái Expo / React Native.

Tuy nhiên, theo phạm vi triển khai được mô tả trong README của dự án, phiên bản Android và iOS hiện sử dụng dữ liệu giả lập cục bộ thay vì kết nối trực tiếp đến MQTT Broker để tiếp nhận dữ liệu từ ESP32.

Vì vậy, chức năng giám sát dữ liệu thực tế thông qua MQTT được tập trung triển khai và kiểm chứng trên phiên bản Web. Phiên bản Android và iOS chủ yếu phục vụ việc mô phỏng giao diện và trải nghiệm sử dụng ứng dụng trên thiết bị di động.

2.4. Các giới hạn của đề tài

Mặc dù hệ thống đáp ứng được các mục tiêu cơ bản về giám sát và cảnh báo kho lạnh, đề tài vẫn tồn tại một số giới hạn nhất định do phạm vi triển khai chủ yếu dựa trên môi trường mô phỏng.

Cụ thể:

- Hệ thống chưa được triển khai trên phần cứng ESP32 thực tế nên chưa đánh giá được đầy đủ độ ổn định khi hoạt động trong môi trường kho lạnh.
- Các cảm biến và thiết bị đầu ra được mô phỏng trên Wokwi nên kết quả có thể có sự khác biệt so với thiết bị vật lý.
- Hệ thống chưa tích hợp thiết bị điều khiển máy nén lạnh hoặc cơ chế tự động điều chỉnh nhiệt độ.
- Ngưỡng nhiệt độ -15,0 °C được thiết lập phục vụ mô hình kho đông trong đề tài, không đại diện cho tiêu chuẩn bảo quản của tất cả các loại thực phẩm, vắc-xin hoặc dược phẩm.
- Chức năng theo dõi cửa tập trung vào việc phát hiện trạng thái mở liên tục, chưa triển khai các cơ chế phân tích hành vi hoặc dự đoán sự cố.
- Phiên bản Android và iOS hiện sử dụng dữ liệu giả lập cục bộ, chưa đồng bộ dữ liệu thực tế từ MQTT Broker như phiên bản Web.

Những giới hạn trên là cơ sở để xác định các hướng phát triển và hoàn thiện hệ thống trong những giai đoạn tiếp theo.

2.5. Kết quả mong đợi

Sau khi hoàn thành, đề tài kỳ vọng xây dựng được một mô hình IoT giám sát kho lạnh có khả năng hoạt động xuyên suốt từ khâu thu thập dữ liệu đến hiển thị thông tin trên ứng dụng web.

Các kết quả cụ thể dự kiến đạt được bao gồm:

1. Xây dựng thành công sơ đồ mạch mô phỏng ESP32 và các cảm biến, thiết bị ngoại vi trên Wokwi.
2. Hoàn thiện chương trình điều khiển ESP32 với các chức năng đo nhiệt độ, độ ẩm, theo dõi cửa và điều khiển thiết bị cảnh báo.
3. Triển khai thành công kết nối MQTT/TLS giữa ESP32 và EMQX Cloud Serverless thông qua cổng 8883.
4. Xây dựng được cơ chế truyền dữ liệu telemetry và cảnh báo theo định dạng JSON.
5. Hoàn thiện giao diện Dashboard trên nền tảng Web với khả năng tiếp nhận và hiển thị dữ liệu từ MQTT Broker.
6. Kiểm thử được các tình huống bất thường như nhiệt độ vượt ngưỡng và cửa kho mở quá 180 giây.
7. Chứng minh được khả năng phối hợp giữa phần cứng mô phỏng, nền tảng Cloud và ứng dụng giám sát trong một hệ thống IoT thống nhất.

Thông qua các kết quả trên, đề tài không chỉ minh họa được nguyên lý hoạt động của một hệ thống giám sát kho lạnh mà còn giúp kiểm chứng tính khả thi của việc ứng dụng công nghệ IoT vào hoạt động theo dõi và cảnh báo môi trường bảo quản từ xa.

CHƯƠNG 3: THÀNH PHẦN PHẦN CỨNG VÀ SƠ ĐỒ CHÂN

3.1. Tổng quan về phần cứng hệ thống

Hệ thống giám sát kho lạnh được xây dựng dựa trên vi điều khiển ESP32, đóng vai trò là bộ xử lý trung tâm, chịu trách nhiệm thu thập dữ liệu từ các cảm biến, tiếp nhận tín hiệu từ nút nhấn, xử lý các điều kiện hoạt động và điều khiển các thiết bị đầu ra.

Mô hình phần cứng bao gồm một vi điều khiển ESP32, hai cảm biến môi trường, một nút nhấn mô phỏng trạng thái cửa kho và ba thiết bị đầu ra phục vụ chức năng chiếu sáng và cảnh báo.

Cụ thể, các thành phần được sử dụng trong hệ thống bao gồm:

- Vi điều khiển ESP32: Thực hiện xử lý dữ liệu, điều khiển các thiết bị ngoại vi và kết nối Wi-Fi để truyền dữ liệu lên máy chủ MQTT.
- Cảm biến DHT22: Thu thập thông tin về nhiệt độ và độ ẩm bên trong kho lạnh.
- Cảm biến quang trở LDR: Đo cường độ ánh sáng môi trường thông qua tín hiệu analog.
- Nút nhấn (Push Button): Mô phỏng trạng thái mở và đóng cửa kho lạnh.
- LED cảnh báo: Phát tín hiệu ánh sáng khi hệ thống phát hiện điều kiện bất thường.
- Còi Buzzer: Phát tín hiệu âm thanh để cảnh báo người dùng khi có sự cố.
- Đèn kho: Mô phỏng hệ thống chiếu sáng bên trong kho, được điều khiển dựa trên trạng thái cửa.

Các linh kiện được kết nối với ESP32 thông qua những chân GPIO phù hợp với chức năng đầu vào, đầu ra và khả năng đọc tín hiệu analog. Việc phân chia chân kết nối được thực hiện nhằm đảm bảo các thành phần có thể hoạt động độc lập và phối hợp với nhau trong quá trình vận hành hệ thống.

3.2. Bộ điều khiển trung tâm ESP32

ESP32 là vi điều khiển được sử dụng làm bộ xử lý trung tâm của hệ thống. Thiết bị tích hợp khả năng kết nối Wi-Fi, hỗ trợ nhiều chân GPIO và các giao tiếp ngoại vi, phù hợp với những ứng dụng giám sát và điều khiển IoT.

Trong đề tài, ESP32 đảm nhiệm các chức năng chính sau:

- Đọc dữ liệu nhiệt độ và độ ẩm từ cảm biến DHT22.
- Đọc giá trị ánh sáng từ cảm biến LDR thông qua bộ chuyển đổi ADC.
- Nhận biết trạng thái đóng và mở cửa thông qua nút nhấn.
- Tính toán thời gian cửa kho mở liên tục.
- So sánh dữ liệu cảm biến với các ngưỡng đã được cấu hình.
- Điều khiển LED cảnh báo, còi Buzzer và đèn chiếu sáng.
- Kết nối mạng Wi-Fi và truyền dữ liệu đến MQTT Broker thông qua giao thức MQTT/TLS.

Việc sử dụng ESP32 giúp tích hợp các chức năng thu thập dữ liệu, xử lý tín hiệu, điều khiển thiết bị và truyền thông mạng trong cùng một bộ điều khiển, góp phần đơn giản hóa kiến trúc phần cứng của hệ thống.

3.3. Các cảm biến đầu vào

DHT22 là cảm biến kỹ thuật số có khả năng đo nhiệt độ và độ ẩm tương đối của môi trường. Cảm biến được sử dụng để theo dõi điều kiện bảo quản bên trong kho lạnh.

Trong hệ thống, DHT22 được kết nối với ESP32 thông qua chân GPIO 15. Cảm biến sử dụng giao thức truyền dữ liệu một dây (single-wire digital communication) để gửi thông tin nhiệt độ và độ ẩm đến bộ điều khiển.

Sau khi nhận được dữ liệu, ESP32 tiến hành xử lý và kiểm tra giá trị nhiệt độ so với ngưỡng an toàn đã được thiết lập ở mức -15,0 °C. Nếu nhiệt độ cao hơn ngưỡng này, hệ thống sẽ xác định trạng thái bất thường và kích hoạt cơ chế cảnh báo.

Bên cạnh đó, giá trị độ ẩm cũng được thu thập và truyền lên ứng dụng web để phục vụ việc giám sát môi trường kho lạnh.

Thông số kết nối:

Thuộc tính	**Giá trị**
Loại cảm biến	DHT22

Chân dữ liệu	GPIO 15
Chế độ	Digital Input
Giao tiếp	Single-wire
Dữ liệu đầu ra	Nhiệt độ, độ ẩm
Vai trò	Giám sát môi trường

LDR (Light Dependent Resistor) là linh kiện điện trở phụ thuộc vào cường độ ánh sáng chiếu vào bề mặt. Điện trở của LDR thay đổi theo mức độ chiếu sáng của môi trường.

Trong đề tài, LDR được sử dụng để mô phỏng chức năng giám sát ánh sáng bên trong kho lạnh. Cảm biến được kết nối với chân GPIO 34 của ESP32, đây là chân hỗ trợ đọc tín hiệu analog thông qua bộ chuyển đổi ADC.

Khi cường độ ánh sáng thay đổi, giá trị điện áp tại đầu vào analog cũng thay đổi theo. ESP32 đọc giá trị này để xác định mức ánh sáng tương đối trong môi trường kho.

Dữ liệu từ LDR có thể được sử dụng để hỗ trợ chức năng điều khiển chiếu sáng, kết hợp với trạng thái cửa kho nhằm mô phỏng hoạt động bật và tắt đèn tự động.

Thông số kết nối:

Thuộc tính	**Giá trị**
Loại cảm biến	LDR

Chân kết nối	GPIO 34
Chế độ	Analog Input
Giao tiếp	ADC
Dữ liệu đầu ra	Giá trị ánh sáng tương đối
Vai trò	Giám sát cường độ ánh sáng

3.4. Thiết bị đầu vào mô phỏng trạng thái cửa kho

Nút nhấn là linh kiện đầu vào được sử dụng để mô phỏng thao tác mở và đóng cửa kho lạnh trong môi trường Wokwi.

Nút nhấn được kết nối với chân GPIO 14 của ESP32 và được cấu hình ở chế độ `INPUT_PULLUP`. Với cách cấu hình này, chân GPIO được kéo lên mức logic HIGH khi nút không được nhấn.

Khi người dùng nhấn nút, chân GPIO được nối xuống GND và chuyển sang mức logic LOW. Chương trình sử dụng sự thay đổi mức logic này để xác định trạng thái cửa kho.

Quy ước hoạt động được thiết lập như sau:

Trạng thái nút nhấn	**Mức logic GPIO 14**	**Trạng thái cửa**
Không nhấn	HIGH	Đóng

Nhấn	LOW	Mở

Khi phát hiện cửa chuyển sang trạng thái mở, ESP32 bắt đầu tính toán thời gian mở cửa liên tục. Nếu thời gian mở vượt quá 180 giây, hệ thống sẽ kích hoạt cảnh báo bằng LED và Buzzer.

Việc sử dụng nút nhấn giúp mô phỏng trực quan các tình huống đóng/mở cửa, thuận tiện cho việc kiểm thử chương trình mà không cần lắp đặt công tắc cửa thực tế.

3.5. Các thiết bị đầu ra

LED được sử dụng để phát tín hiệu ánh sáng nhằm thông báo cho người dùng khi hệ thống phát hiện các điều kiện bất thường trong kho lạnh.

LED cảnh báo được kết nối với chân GPIO 2 của ESP32 và được cấu hình ở chế độ OUTPUT.

Khi nhiệt độ trong kho vượt quá ngưỡng -15,0 °C hoặc thời gian cửa mở liên tục vượt quá 180 giây, ESP32 sẽ xuất tín hiệu điều khiển để bật LED. Khi các điều kiện cảnh báo không còn tồn tại, LED sẽ được điều khiển trở về trạng thái tắt theo logic chương trình.

Thiết bị này giúp người dùng dễ dàng nhận biết trạng thái bất thường thông qua tín hiệu ánh sáng trực tiếp tại khu vực kho.

Buzzer là thiết bị phát âm thanh, được sử dụng để tạo tín hiệu cảnh báo khi phát hiện sự cố hoặc điều kiện vận hành không an toàn.

Buzzer được kết nối với chân GPIO 4 của ESP32 và được cấu hình làm chân đầu ra. Trong mô hình, còi được lập trình phát âm thanh với tần số 1000 Hz khi hệ thống xuất hiện cảnh báo.

Cụ thể, khi nhiệt độ vượt quá giới hạn an toàn hoặc cửa kho mở liên tục quá 180 giây, ESP32 sẽ kích hoạt Buzzer để phát âm thanh cảnh báo.

Việc kết hợp Buzzer với LED giúp nâng cao khả năng nhận biết sự cố, đặc biệt trong những trường hợp người quản lý không trực tiếp quan sát màn hình Dashboard.

Đèn kho được mô phỏng bằng LED hoặc mô-đun đèn tương ứng, đóng vai trò là hệ thống chiếu sáng bên trong kho lạnh.

Đèn kho được kết nối với chân GPIO 18 của ESP32 và hoạt động ở chế độ OUTPUT.

Chức năng của thiết bị là tự động bật sáng khi cửa kho được mở và tắt khi cửa kho được đóng lại. Cơ chế này mô phỏng hệ thống chiếu sáng tự động thường được sử dụng trong các kho bảo quản.

Việc điều khiển đèn dựa trên trạng thái cửa giúp hạn chế tình trạng đèn hoạt động không cần thiết, góp phần tiết kiệm điện năng trong quá trình vận hành.

3.6. Bảng phân công chân kết nối ESP32

Dựa trên các khai báo chân trong chương trình điều khiển, hệ thống sử dụng các chân GPIO của ESP32 để kết nối với cảm biến, nút nhấn và thiết bị đầu ra.

Bảng 3.1. Bảng phân công chân kết nối phần cứng

STT	**Thành phần**	**Chân ESP32**	**Chế độ hoạt động**	**Chức năng**
1	DHT22	GPIO 15	Digital Input	Đo nhiệt độ, độ ẩm
2	Nút nhấn	GPIO 14	INPUT_PULLUP	Mô phỏng cửa kho
3	LDR	GPIO 34	Analog Input (ADC)	Đo cường độ ánh sáng
4	LED cảnh báo	GPIO 2	OUTPUT	Cảnh báo bằng ánh sáng
5	Buzzer	GPIO 4	OUTPUT	Cảnh báo bằng âm thanh 1000 Hz

6	Đèn kho	GPIO 18	OUTPUT	Điều khiển chiếu sáng

Nhìn chung, việc phân công chân kết nối được thực hiện theo chức năng của từng linh kiện. Các chân GPIO 14 và GPIO 34 được sử dụng để tiếp nhận tín hiệu đầu vào, GPIO 15 phục vụ giao tiếp với cảm biến DHT22, trong khi GPIO 2, GPIO 4 và GPIO 18 đảm nhiệm chức năng điều khiển thiết bị đầu ra.

Đặc biệt, GPIO 34 là chân chỉ hỗ trợ đầu vào trên ESP32 nên phù hợp với chức năng đọc tín hiệu analog từ cảm biến LDR, không được sử dụng để điều khiển thiết bị đầu ra.

3.7. Kết nối Wi-Fi và các thư viện sử dụng

Để thực hiện việc truyền dữ liệu lên nền tảng đám mây, ESP32 cần được kết nối với mạng Wi-Fi. Trong phạm vi đề tài, do phần cứng được mô phỏng trên nền tảng Wokwi nên hệ thống sử dụng mạng Wi-Fi ảo có tên `Wokwi-GUEST`.

Các thông số kết nối được thiết lập như sau:

Thông số	**Giá trị**
Tên mạng Wi-Fi (SSID)	Wokwi-GUEST
Mật khẩu	Không yêu cầu
Kênh Wi-Fi	6
Thiết bị kết nối	ESP32

Mục đích	Kết nối Internet và truyền dữ liệu MQTT

Mạng Wokwi-GUEST cho phép ESP32 trong môi trường mô phỏng truy cập Internet mà không cần cấu hình mạng Wi-Fi vật lý. Nhờ đó, thiết bị có thể kết nối đến EMQX Cloud để gửi dữ liệu cảm biến và trạng thái hoạt động của hệ thống.

Để triển khai các chức năng thu thập dữ liệu, kết nối mạng và truyền thông MQTT, chương trình ESP32 sử dụng một số thư viện hỗ trợ.

STT	Thư viện	Chức năng
1	`WiFi.h`	Quản lý và thiết lập kết nối Wi-Fi
2	`WiFiClientSecure.h`	Hỗ trợ kết nối mạng có mã hóa TLS
3	`PubSubClient.h`	Thực hiện giao tiếp MQTT, gửi và nhận dữ liệu
4	`DHT.h`	Đọc dữ liệu nhiệt độ và độ ẩm từ DHT22

Bảng 3.7.3. Các thư viện được sử dụng

Vai trò cụ thể của từng thư viện:

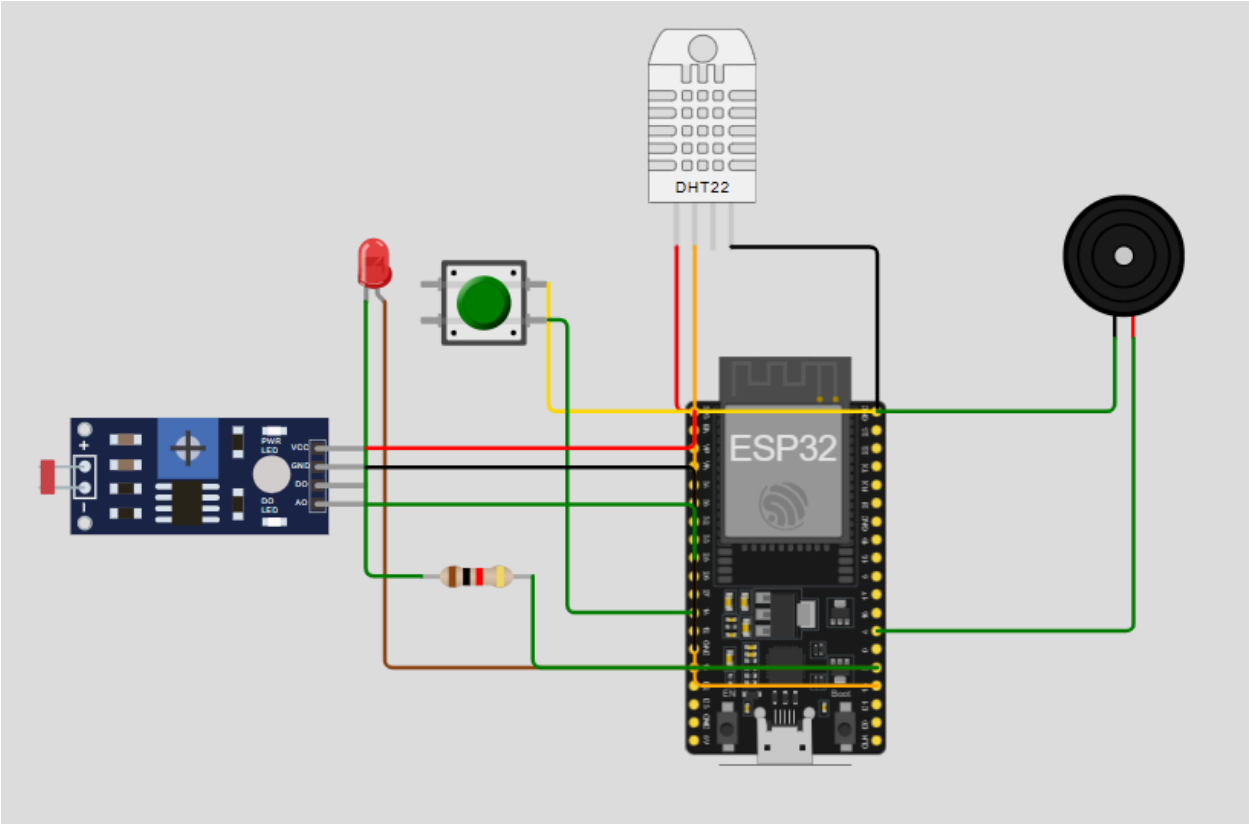
- WiFi.h: Cung cấp các hàm hỗ trợ ESP32 kết nối đến mạng Wi-Fi, kiểm tra trạng thái kết nối và quản lý quá trình truy cập Internet.
- WiFiClientSecure.h: Hỗ trợ tạo kết nối TCP có mã hóa TLS, được sử dụng làm lớp kết nối bảo mật khi giao tiếp với EMQX Cloud.
- PubSubClient.h: Cung cấp các chức năng kết nối MQTT Broker, đăng ký Topic và gửi dữ liệu lên máy chủ theo mô hình Publish/Subscribe.

- DHT.h: Hỗ trợ giao tiếp với cảm biến DHT22, đọc và xử lý dữ liệu nhiệt độ, độ ẩm.

Việc kết hợp các thư viện trên giúp chương trình ESP32 thực hiện đầy đủ các chức năng từ đọc cảm biến, điều khiển thiết bị đến kết nối và truyền dữ liệu lên máy chủ đám mây.

3.8. Sơ đồ kết nối phần cứng

Sơ đồ kết nối phần cứng thể hiện mối liên hệ giữa ESP32 với các cảm biến đầu vào, nút nhấn và thiết bị đầu ra. Đây là cơ sở để xây dựng mạch mô phỏng trên Wokwi và triển khai chương trình điều khiển.



Hình 3.8.1. Sơ đồ kết nối phần cứng hệ thống giám sát kho lạnh.

Nguyên lý kết nối tổng quát được mô tả như sau:

- Cảm biến DHT22 kết nối với GPIO 15 để truyền dữ liệu nhiệt độ và độ ẩm.
- Nút nhấn kết nối với GPIO 14 và được cấu hình điện trở kéo lên nội bộ để mô phỏng trạng thái cửa kho.
- Cảm biến LDR kết nối với GPIO 34 để cung cấp tín hiệu analog cho ESP32.

- LED cảnh báo kết nối với GPIO 2 để phát tín hiệu ánh sáng khi có sự cố.
- Buzzer kết nối với GPIO 4 để phát tín hiệu âm thanh cảnh báo.
- Đèn kho kết nối với GPIO 18 để mô phỏng hoạt động chiếu sáng tự động.

Các linh kiện được cấp nguồn và kết nối GND phù hợp với yêu cầu hoạt động của từng thành phần. Đối với cảm biến LDR, cần sử dụng mạch phân áp để tạo tín hiệu điện áp analog đưa vào chân GPIO 34.

Chương 4: KIẾN TRÚC HỆ THỐNG

4.1. Tổng quan kiến trúc

Hệ thống giám sát kho lạnh được xây dựng theo mô hình IoT kết hợp nền tảng điện toán đám mây, gồm năm thành phần chính: ESP32, EMQX Cloud, Backend Server Node.js, ứng dụng Web/Mobile và Firebase Cloud Messaging (FCM).

ESP32 đóng vai trò thu thập dữ liệu từ các cảm biến, xử lý trạng thái cửa kho và thực hiện cảnh báo tại chỗ thông qua LED, buzzer và đèn kho. Sau khi xử lý, ESP32 gửi dữ liệu lên EMQX Cloud thông qua giao thức MQTT kết hợp TLS nhằm bảo vệ dữ liệu trong quá trình truyền tải.

EMQX Cloud đóng vai trò là MQTT Broker trung gian, tiếp nhận các bản tin từ ESP32 và chuyển tiếp đến những thành phần đã đăng ký theo dõi topic tương ứng. Backend Server được xây dựng bằng Node.js sẽ kết nối đến EMQX để nhận dữ liệu, kiểm tra nội dung bản tin, xử lý các điều kiện cảnh báo và phối hợp với Firebase Cloud Messaging để gửi thông báo đẩy đến thiết bị di động.

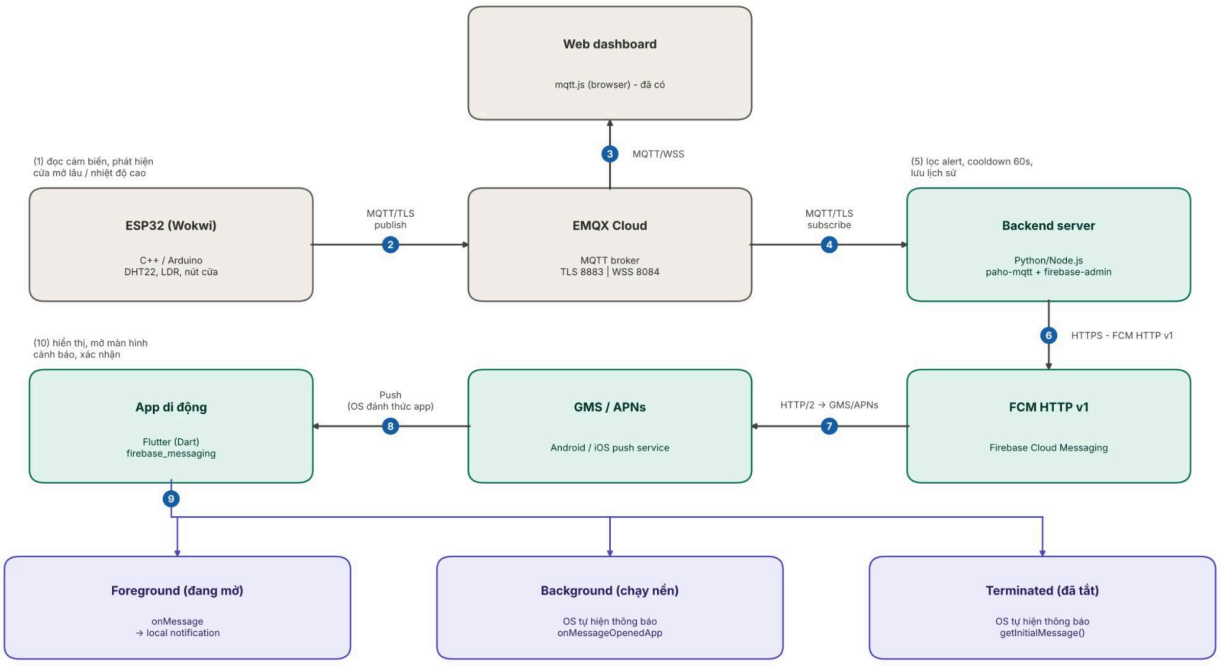
Ứng dụng Web có thể nhận dữ liệu trực tiếp từ EMQX thông qua WebSocket bảo mật để cập nhật giao diện giám sát. Trong khi đó, ứng dụng Mobile sử dụng Firebase Cloud Messaging để nhận thông báo khi hệ thống phát hiện các tình huống bất thường. Như vậy, hệ thống hỗ trợ cả việc theo dõi trạng thái kho lạnh trên giao diện và thông báo cho người dùng khi có sự cố.

4.2. Mô hình truyền dữ liệu

Hệ thống sử dụng mô hình truyền dữ liệu theo chiều từ thiết bị IoT đến nền tảng đám mây và ứng dụng người dùng. ESP32 là bên duy nhất thực hiện thao tác Publish dữ liệu cảm biến lên MQTT Broker. Backend Node.js và ứng dụng Web đóng vai trò Subscribe để tiếp nhận dữ liệu từ các topic đã đăng ký.

Quy trình truyền dữ liệu được thực hiện như sau:

1. Thu thập dữ liệu: ESP32 đọc nhiệt độ, độ ẩm từ cảm biến DHT22, giá trị ánh sáng từ LDR và trạng thái cửa từ nút nhấn.
2. Xử lý tại thiết bị: ESP32 kiểm tra các điều kiện hoạt động, xác định trạng thái cửa, thời gian mở cửa và điều kiện cảnh báo. Nếu phát hiện bất thường, thiết bị kích hoạt LED và buzzer.
3. Gửi dữ liệu lên EMQX: ESP32 đóng gói dữ liệu thành JSON và Publish lên các MQTT topic thông qua kết nối MQTT/TLS.
4. Tiếp nhận tại Backend: Backend Node.js Subscribe các topic tương ứng, tiếp nhận bản tin và kiểm tra cấu trúc dữ liệu.
5. Xử lý cảnh báo: Backend phân tích dữ liệu, xác định các sự kiện cần thông báo và thực hiện xử lý theo điều kiện đã thiết lập.
6. Gửi thông báo đẩy: Khi phát hiện sự kiện cần thông báo, Backend sử dụng Firebase Admin SDK để gửi yêu cầu đến Firebase Cloud Messaging.
7. Hiển thị thông báo: Firebase Cloud Messaging chuyển thông báo đến ứng dụng Mobile đã đăng ký nhận thông báo.
8. Hiển thị trên Web: Ứng dụng Web Subscribe MQTT thông qua WebSocket bảo mật và cập nhật dữ liệu lên Dashboard.



Hình 4.2.1. Kiến trúc tổng thể của hệ thống giám sát kho lạnh

4.3. Chức năng của từng thành phần

4.3.1. ESP32 – Thiết bị thu thập dữ liệu

ESP32 là thành phần trực tiếp tương tác với các cảm biến và thiết bị ngoại vi trong mô hình kho lạnh. Thiết bị đảm nhiệm các chức năng:

- Đọc nhiệt độ và độ ẩm từ DHT22.
- Đọc giá trị ánh sáng từ cảm biến LDR.
- Xác định trạng thái cửa kho thông qua nút nhấn.
- Tính toán thời gian cửa mở liên tục.
- Điều khiển đèn kho theo trạng thái cửa.
- Kích hoạt LED và buzzer khi xảy ra điều kiện cảnh báo.
- Đóng gói dữ liệu thành JSON và Publish lên EMQX Cloud bằng MQTT/TLS.

Trong kiến trúc hệ thống, ESP32 chỉ đóng vai trò gửi dữ liệu, không nhận lệnh điều khiển từ ứng dụng Web hoặc Backend.

4.3.2. EMQX Cloud – Máy chủ MQTT Broker

EMQX Cloud là nền tảng trung gian tiếp nhận và phân phối dữ liệu giữa ESP32, Backend Node.js và ứng dụng Web.

Các nhiệm vụ chính gồm:

- Tiếp nhận bản tin MQTT từ ESP32.
- Quản lý các kết nối MQTT của thiết bị và Backend.
- Chuyển tiếp bản tin đến các Client đã đăng ký Subscribe topic tương ứng.
- Hỗ trợ giao tiếp bảo mật thông qua TLS.
- Phân phối dữ liệu đến ứng dụng Web qua MQTT over WebSocket Secure (WSS).

Việc sử dụng EMQX giúp tách biệt thiết bị IoT khỏi các ứng dụng sử dụng dữ liệu, đồng thời tạo điều kiện mở rộng hệ thống khi cần bổ sung thêm thiết bị hoặc dịch vụ xử lý.

4.3.3. Backend Server Node.js

Backend Server được xây dựng bằng Node.js, đóng vai trò là tầng xử lý nghiệp vụ trung gian giữa MQTT Broker và dịch vụ thông báo Firebase.

Backend kết nối đến EMQX Cloud bằng MQTT/TLS và đăng ký nhận dữ liệu từ các topic do ESP32 Publish. Khi nhận được bản tin, Backend thực hiện các công việc:

- Kiểm tra tính hợp lệ của dữ liệu JSON.
- Phân loại bản tin thành dữ liệu giám sát (telemetry) và dữ liệu cảnh báo (alert).
- Phân tích các thông số nhiệt độ, độ ẩm và trạng thái cửa.
- Xác định sự kiện cần gửi thông báo đến người dùng.
- Gọi Firebase Admin SDK để gửi thông báo đẩy thông qua FCM.
- Có thể mở rộng để lưu trữ dữ liệu vào cơ sở dữ liệu và cung cấp API cho ứng dụng.

Backend giúp tập trung xử lý các nghiệp vụ liên quan đến cảnh báo, hạn chế việc đặt toàn bộ logic thông báo trực tiếp trong firmware ESP32.

4.3.4. Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) là dịch vụ hỗ trợ gửi thông báo đẩy đến các thiết bị di động. Trong hệ thống, FCM được sử dụng để chuyển các thông báo do Backend tạo ra đến ứng dụng Mobile.

Quy trình gửi thông báo gồm:

1. Ứng dụng Mobile đăng ký nhận thông báo và lấy FCM Registration Token.
2. Token được cung cấp cho Backend để quản lý thiết bị nhận thông báo.
3. Khi Backend phát hiện sự kiện cảnh báo, Backend tạo nội dung thông báo.
4. Backend sử dụng Firebase Admin SDK gửi thông báo đến FCM.
5. FCM chuyển thông báo đến thiết bị tương ứng.
6. Ứng dụng Mobile hiển thị nội dung cảnh báo cho người dùng.

Một số loại thông báo có thể được triển khai:

Sự kiện	**Nội dung thông báo**
Nhiệt độ vượt ngưỡng	Cảnh báo: Nhiệt độ kho lạnh vượt mức cho phép
Cửa mở quá lâu	Cảnh báo: Cửa kho đã mở liên tục quá 3 phút

Nhiệt độ trở lại bình thường	Thông báo: Nhiệt độ kho đã ổn định
Cửa kho đóng lại	Thông báo: Cửa kho đã được đóng

Việc gửi thông báo cần được kiểm soát để tránh gửi lặp liên tục khi một điều kiện cảnh báo vẫn đang tồn tại. Backend có thể sử dụng cơ chế phát hiện thời điểm chuyển trạng thái hoặc giới hạn tần suất gửi thông báo.

4.3.5. Ứng dụng Web và Mobile

Ứng dụng người dùng được xây dựng bằng Expo và React Native, hỗ trợ tổ chức giao diện thành nhiều màn hình chức năng.

Đối với phiên bản Web, ứng dụng sử dụng MQTT over WebSocket Secure để kết nối đến EMQX Cloud. Sau khi kết nối thành công, ứng dụng đăng ký các topic cần theo dõi, tiếp nhận bản tin và cập nhật dữ liệu trên Dashboard.

Đối với phiên bản Android/iOS, theo phạm vi triển khai hiện tại, ứng dụng chưa kết nối trực tiếp đến MQTT Broker. File `mqttClient.ts` đang là phiên bản rỗng, do đó dữ liệu hiển thị trên hai nền tảng này được tạo từ nguồn mô phỏng cục bộ. Khi tích hợp FCM, ứng dụng Mobile có thể nhận thông báo đẩy do Backend gửi qua Firebase.

4.4. Luồng xử lý cảnh báo và thông báo đẩy

Khi hệ thống phát hiện nhiệt độ vượt quá ngưỡng -15 °C hoặc cửa kho mở liên tục quá 180 giây, ESP32 thực hiện cảnh báo tại chỗ bằng LED và buzzer. Đồng thời, thiết bị gửi bản tin lên EMQX Cloud để các thành phần khác tiếp nhận.

Backend Node.js nhận bản tin và xác định loại sự kiện. Nếu sự kiện thỏa mãn điều kiện gửi thông báo, Backend tạo nội dung cảnh báo và gửi đến Firebase Cloud Messaging. Sau đó, FCM chuyển thông báo đến ứng dụng Mobile của người dùng.

Quy trình này tạo ra hai hình thức cảnh báo song song:

- Cảnh báo tại chỗ: ESP32 điều khiển LED và buzzer để người có mặt tại kho nhận biết sự cố.
- Cảnh báo từ xa: Backend phối hợp với Firebase Cloud Messaging để gửi thông báo đến điện thoại người dùng.

4.5. Phạm vi triển khai và giới hạn nền tảng

Trong phiên bản hiện tại, hệ thống có sự khác biệt giữa các nền tảng triển khai:

Nền tảng	Nguồn dữ liệu	Chức năng
ESP32 trên Wokwi	Cảm biến mô phỏng	Thu thập, xử lý và gửi dữ liệu MQTT
EMQX Cloud	Bản tin MQTT từ ESP32	Tiếp nhận và phân phối dữ liệu
Backend Node.js	Dữ liệu từ EMQX	Xử lý nghiệp vụ và gửi thông báo FCM
Web	MQTT qua WSS	Nhận dữ liệu thật và cập nhật Dashboard
Android/iOS	Dữ liệu mô phỏng cục bộ	Hiển thị giao diện và có thể nhận thông báo FCM khi tích hợp hoàn chỉnh

Như vậy, phiên bản Web là nền tảng trực tiếp nhận dữ liệu thực từ mô hình ESP32 trên Wokwi. Phiên bản Android/iOS hiện sử dụng dữ liệu mô phỏng cục bộ, không đồng bộ telemetry trực tiếp với thiết bị. Việc tích hợp Firebase Cloud Messaging bổ sung khả năng thông báo từ xa, nhưng không đồng nghĩa với việc Mobile đã có chức năng giám sát dữ liệu MQTT theo thời gian thực.

CHƯƠNG 5: THIẾT KẾ PHẦN MỀM VÀ THUẬT TOÁN

5.1. Tổng quan thiết kế phần mềm

Phần mềm của hệ thống giám sát kho lạnh được chia thành ba thành phần chính: chương trình điều khiển trên ESP32, ứng dụng Web và Backend Server Node.js kết hợp Firebase Cloud Messaging.

- Firmware ESP32: Thu thập dữ liệu cảm biến, xử lý trạng thái cửa, điều khiển các thiết bị đầu ra và gửi dữ liệu lên EMQX Cloud thông qua MQTT.
- Ứng dụng Web: Tiếp nhận dữ liệu MQTT qua WebSocket bảo mật, cập nhật trạng thái giám sát, lưu lịch sử và tạo cảnh báo trên giao diện.
- Backend Node.js và Firebase: Tiếp nhận dữ liệu từ MQTT Broker, xử lý sự kiện cảnh báo và gửi thông báo đẩy đến ứng dụng Mobile thông qua Firebase Cloud Messaging.

Trong đó, ESP32 đảm nhiệm việc xử lý trực tiếp tại thiết bị, ứng dụng Web đảm nhiệm việc hiển thị và thống kê dữ liệu, còn Backend đảm nhiệm việc xử lý các nghiệp vụ phía máy chủ và thông báo từ xa.

5.2. Thiết kế thuật toán chương trình ESP32

Chương trình firmware được triển khai trong tệp `arduino/ArduinoCode.ino` bằng ngôn ngữ C++ trên nền tảng Arduino Framework. Chương trình sử dụng hàm `setup()` để khởi tạo các thiết bị ngoại vi, kết nối Wi-Fi và thiết lập MQTT. Sau đó, hàm `loop()` được thực thi liên tục để thu thập dữ liệu, xử lý điều kiện và truyền thông tin lên máy chủ.

Chu kỳ gửi dữ liệu telemetry được thiết lập là 5 giây. Trong mỗi chu kỳ, ESP32 thực hiện lần lượt các bước xử lý.

5.2.1. Khởi tạo hệ thống

Khi khởi động, ESP32 thực hiện các công việc:

- Khởi tạo Serial để phục vụ quá trình theo dõi và kiểm tra chương trình.
- Khởi tạo cảm biến DHT22.
- Cấu hình GPIO 14 ở chế độ `INPUT_PULLUP` để đọc trạng thái nút nhấn.
- Cấu hình GPIO 34 ở chế độ đầu vào Analog để đọc giá trị LDR.
- Cấu hình GPIO 2, GPIO 4 và GPIO 18 làm đầu ra điều khiển LED cảnh báo, buzzer và đèn kho.
- Kết nối mạng Wi-Fi Wokwi-GUEST.
- Thiết lập kết nối MQTT/TLS đến EMQX Cloud.

5.2.2. Duy trì kết nối MQTT

Trong mỗi vòng lặp, chương trình kiểm tra trạng thái kết nối đến MQTT Broker. Nếu kết nối bị gián đoạn, ESP32 gọi hàm `reconnect()` để thực hiện kết nối lại.

Quá trình kết nối sử dụng Client ID ngẫu nhiên nhằm hạn chế xung đột định danh giữa các phiên kết nối. Nếu kết nối thất bại, chương trình chờ khoảng 5 giây trước khi thử lại.

Sau khi kiểm tra kết nối, chương trình gọi `client.loop()` để duy trì hoạt động của MQTT Client và xử lý các tác vụ mạng cần thiết.

5.2.3. Đọc dữ liệu cảm biến

ESP32 tiến hành đọc giá trị ánh sáng từ cảm biến LDR thông qua chân GPIO 34 bằng hàm `analogRead(34)`. Giá trị ADC được in ra Serial để phục vụ theo dõi trong quá trình mô phỏng.

Tiếp theo, chương trình đọc nhiệt độ và độ ẩm từ cảm biến DHT22. Nếu kết quả trả về không hợp lệ (`NaN`), chương trình chờ 2 giây và bỏ qua chu kỳ xử lý hiện tại nhằm tránh sử dụng dữ liệu lỗi để tính toán hoặc gửi lên máy chủ.

Việc kiểm tra dữ liệu cảm biến trước khi xử lý giúp hạn chế các cảnh báo sai do lỗi đọc cảm biến.

5.2.4. Xử lý trạng thái cửa kho

Trạng thái cửa được xác định thông qua nút nhấn nối với GPIO 14 và sử dụng chế độ `INPUT_PULLUP`.

Do đặc điểm của cấu hình này, khi không nhấn nút, chân GPIO có mức logic HIGH; khi nhấn nút, chân GPIO chuyển sang mức LOW. Chương trình sử dụng trạng thái này để xác định cửa đang mở hay đóng.

Khi phát hiện chuyển trạng thái từ đóng sang mở, chương trình ghi nhận thời điểm bắt đầu mở cửa vào biến `doorOpenStart` bằng hàm `millis()`. Đồng thời, biến `alertSent` được đặt lại thành `false` để cho phép phát sinh cảnh báo nếu cửa tiếp tục mở quá thời gian quy định.

Khi phát hiện chuyển trạng thái từ mở sang đóng, chương trình cập nhật trạng thái cửa về đóng. Việc đóng cửa cũng kết thúc chu kỳ theo dõi thời gian mở hiện tại.

5.2.5. Điều khiển thiết bị đầu ra

Sau khi đọc và xử lý dữ liệu, ESP32 xác định trạng thái của các thiết bị đầu ra theo hai nhóm điều kiện chính.

Điều khiển đèn kho:

- Khi cửa mở, GPIO 18 được đặt ở mức HIGH để bật đèn kho.
- Khi cửa đóng, GPIO 18 được đặt ở mức LOW để tắt đèn kho.

Điều khiển LED và buzzer cảnh báo:

- Khi nhiệt độ lớn hơn -15 °C, hệ thống kích hoạt LED và buzzer.
- Khi cửa mở vượt quá 180 giây, hệ thống cũng kích hoạt LED và buzzer.
- Khi không có điều kiện cảnh báo, LED và buzzer được tắt.

Buzzer được cấu hình phát âm thanh ở tần số 1000 Hz. Nhờ đó, hệ thống có thể cảnh báo trực quan và bằng âm thanh khi phát hiện điều kiện bất thường.

5.2.7. Đóng gói và gửi dữ liệu MQTT

Sau khi hoàn tất quá trình xử lý, ESP32 đóng gói dữ liệu thành định dạng JSON và Publish lên các MQTT topic tương ứng.

Bản tin telemetry chứa các thông tin giám sát như:

- Nhiệt độ hiện tại.
- Độ ẩm hiện tại.
- Giá trị ánh sáng từ LDR.
- Trạng thái cửa kho.
- Thời gian cửa mở.
- Trạng thái cảnh báo.

Khi cửa mở quá thời gian quy định, ESP32 gửi thêm một bản tin alert riêng để thông báo sự kiện cửa mở quá lâu.

Sau khi gửi dữ liệu, chương trình thực hiện `delay(5000)` để chờ 5 giây trước khi bắt đầu chu kỳ tiếp theo.

5.3. Bảng điều kiện xử lý và điều khiển

Các điều kiện hoạt động của firmware được tổng hợp trong bảng sau:

Điều kiện	LED và buzzer	Đèn kho	Bản tin MQTT
Nhiệt độ ≤ -15 °C, cửa đóng	Tắt	Tắt	Telemetry mỗi 5 giây
Nhiệt độ > -15 °C	Bật	Theo trạng thái cửa	Telemetry mỗi 5 giây
Cửa mở ≤ 180 giây, nhiệt độ bình thường	Tắt	Bật	Telemetry, `door_open: true`
Cửa mở ≤ 180 giây, nhiệt độ vượt ngưỡng	Bật	Bật	Telemetry, `door_open: true`
Cửa mở > 180 giây	Bật	Bật	Telemetry và một bản tin alert cho mỗi lần mở cửa

5.4. Thiết kế xử lý dữ liệu phía ứng dụng Web

5.4.1. Tiếp nhận dữ liệu MQTT

Ứng dụng Web sử dụng tệp `src/services/mqttClient.web.ts` để kết nối đến EMQX Cloud thông qua giao thức MQTT over WebSocket Secure (WSS).

Sau khi kết nối thành công, ứng dụng đăng ký Subscribe các topic cần thiết để tiếp nhận dữ liệu do ESP32 gửi lên. Khi nhận được bản tin, chương trình thực hiện:

1. Chuyển nội dung bản tin từ chuỗi JSON thành đối tượng dữ liệu.
2. Kiểm tra cấu trúc và các trường dữ liệu cần thiết.
3. Xác định loại bản tin là telemetry hay alert.

4. Chuyển dữ liệu hợp lệ đến `ColdStorageContext.tsx` để cập nhật trạng thái.

Việc kiểm tra dữ liệu trước khi cập nhật giúp hạn chế lỗi khi bản tin không đúng định dạng hoặc thiếu trường thông tin.

5.4.2. Xử lý cảnh báo nhiệt độ

Khi nhận được bản tin telemetry, Web kiểm tra giá trị nhiệt độ hiện tại. Nếu nhiệt độ lớn hơn -15 °C, ứng dụng tạo một cảnh báo nhiệt độ để thông báo trạng thái bất thường.

Để tránh tạo quá nhiều cảnh báo khi nhiệt độ liên tục vượt ngưỡng, hệ thống áp dụng khoảng thời gian giới hạn 60 giây giữa các lần tạo cảnh báo cùng loại.

Điều này giúp danh sách cảnh báo dễ theo dõi hơn và hạn chế tình trạng một sự cố tạo ra quá nhiều bản ghi trùng lặp.

5.4.3. Xử lý cảnh báo cửa mở quá lâu

Khi nhận được bản tin có loại `alert`, ứng dụng Web phân tích nội dung để xác định sự kiện được gửi từ ESP32.

Đối với sự kiện cửa mở quá lâu, Context tạo một bản ghi cảnh báo với các thông tin phù hợp, chẳng hạn như loại cảnh báo, nội dung mô tả và thời điểm phát sinh.

Cảnh báo sau đó được hiển thị trong màn hình quản lý cảnh báo để người dùng có thể theo dõi các sự kiện bất thường đã xảy ra.

5.5. Thiết kế Backend Node.js và Firebase Cloud Messaging

Backend Node.js đảm nhiệm xử lý dữ liệu và sự kiện cảnh báo ở phía máy chủ. Thành phần này Subscribe các topic từ EMQX Cloud, tiếp nhận dữ liệu từ ESP32 và xác định các sự kiện cần gửi thông báo.

Quy trình xử lý gồm:

1. Backend thiết lập kết nối MQTT/TLS đến EMQX Cloud.
2. Đăng ký nhận các topic telemetry và alert.
3. Nhận bản tin và kiểm tra nội dung JSON.
4. Phân loại dữ liệu và xác định điều kiện cảnh báo.
5. Khi phát hiện sự kiện cần thông báo, Backend tạo nội dung thông báo.
6. Sử dụng Firebase Admin SDK gửi thông báo đến Firebase Cloud Messaging.
7. FCM chuyển thông báo đến ứng dụng Mobile đã đăng ký nhận thông báo.

Backend và Web hoạt động độc lập trong việc tiếp nhận dữ liệu từ EMQX. Web phục vụ giám sát trực tiếp, trong khi Backend xử lý nghiệp vụ và gửi thông báo đẩy. Hai thành phần này không yêu cầu Web phải hoạt động thì Backend mới có thể xử lý cảnh báo.

Lưu ý về phạm vi triển khai: Nếu Backend và Firebase chưa được tích hợp hoặc kiểm thử thực tế, nội dung mục này cần được xác định là thiết kế mở rộng. Không nên mô tả chức năng gửi thông báo FCM như một kết quả đã hoàn thành nếu chưa có minh chứng triển khai.

CHƯƠNG 6: GIAO THỨC MQTT VÀ ĐỊNH DẠNG DỮ LIỆU

6.1. Tổng quan giao thức MQTT

Hệ thống sử dụng giao thức MQTT (Message Queuing Telemetry Transport) để trao đổi dữ liệu giữa thiết bị ESP32 và các thành phần thuộc nền tảng giám sát. MQTT là giao thức truyền thông dạng Publish/Subscribe, hoạt động thông qua một máy chủ trung gian gọi là MQTT Broker.

Trong mô hình triển khai, ESP32 và ứng dụng Web đều là Client kết nối đến cùng một MQTT Broker là EMQX Cloud Serverless. ESP32 đảm nhiệm việc gửi dữ liệu bằng thao tác Publish, trong khi ứng dụng Web đăng ký nhận dữ liệu bằng thao tác Subscribe.

Việc sử dụng MQTT giúp giảm độ phức tạp trong giao tiếp giữa thiết bị và ứng dụng. ESP32 không cần biết trực tiếp địa chỉ của ứng dụng Web mà chỉ cần gửi bản tin đến Broker. EMQX sẽ tiếp nhận và chuyển tiếp bản tin đến các Client đã đăng ký theo dõi topic tương ứng.

Bên cạnh đó, hệ thống sử dụng cơ chế mã hóa TLS để bảo vệ dữ liệu trong quá trình truyền tải, hạn chế nguy cơ bị đọc hoặc can thiệp trái phép khi dữ liệu di chuyển qua mạng.

6.2. Mô hình kết nối MQTT

Trong hệ thống, ESP32 và Web sử dụng hai phương thức kết nối khác nhau nhưng cùng truy cập vào một MQTT Broker.

- ESP32: Kết nối đến EMQX Cloud bằng MQTT qua TLS, sử dụng cổng 8883 và thư viện PubSubClient.
- Ứng dụng Web: Kết nối đến EMQX Cloud bằng MQTT over WebSocket Secure (WSS), sử dụng cổng 8084 với đường dẫn `/mqtt` và thư viện mqtt.js.

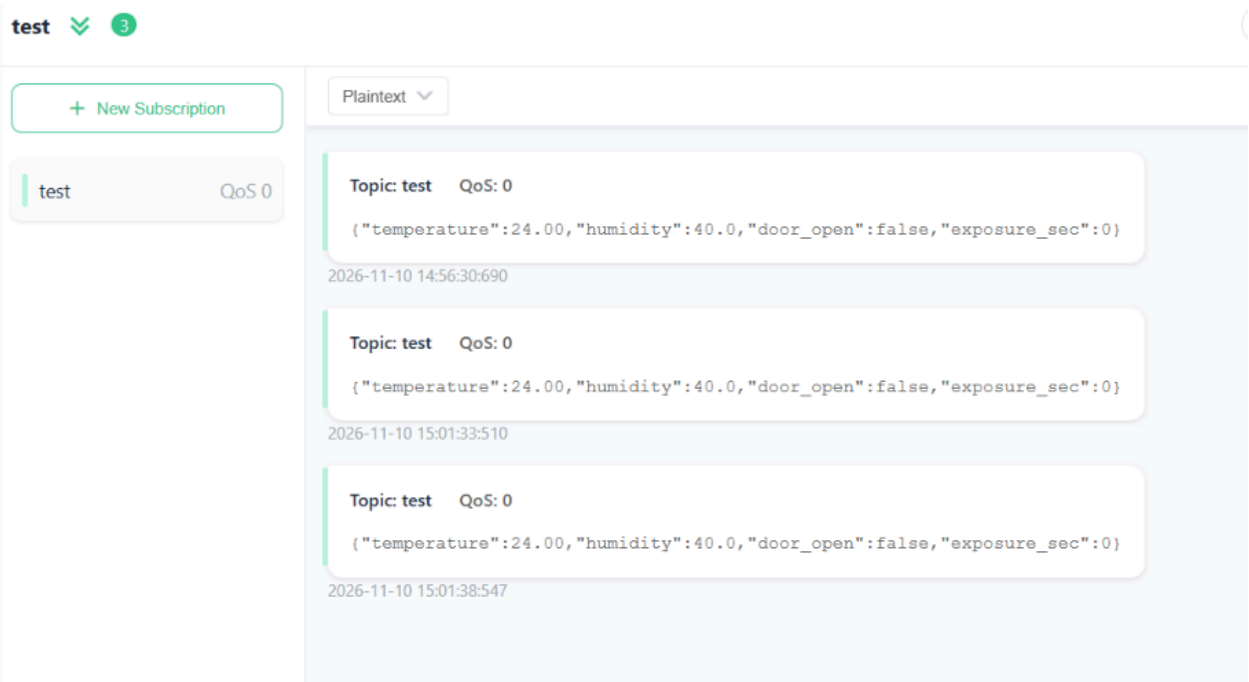
- EMQX Cloud: Tiếp nhận bản tin từ ESP32 và chuyển tiếp đến ứng dụng Web dựa trên topic đã đăng ký.

Thông tin cấu hình kết nối được tổng hợp trong bảng sau:

Bảng 6.1. Thông số kết nối MQTT của hệ thống

Thông số	**ESP32 (Wokwi)**	**Ứng dụng Web**
Giao thức	MQTT qua TLS	MQTT qua WebSocket bảo mật (WSS)
Cổng kết nối	8883	8084
Đường dẫn	Không áp dụng	`/mqtt`
Thư viện	PubSubClient	mqtt.js
Vai trò	Publish	Subscribe
Topic	`test`	`test`
Phương thức xác thực	Username và Password	Username và Password
Nguồn cấu hình	Firmware	Biến môi trường `EXPO_PUBLIC_*`

Mặc dù sử dụng hai giao thức kết nối khác nhau, cả ESP32 và Web đều giao tiếp với cùng một Broker. EMQX chịu trách nhiệm chuyển đổi và phân phối bản tin phù hợp giữa các kết nối MQTT và WebSocket.



Hình 6.1: Dữ liệu truyền trên MQTT

6.3. Kết quả mô phỏng và kịch bản kiểm thử

Các kịch bản dưới đây là hành vi mong đợi suy ra từ mã nguồn. Khi nộp báo cáo, hãy chèn ảnh chụp màn hình Wokwi (mạch, Serial Monitor) và dashboard web vào cột kết quả thực tế.

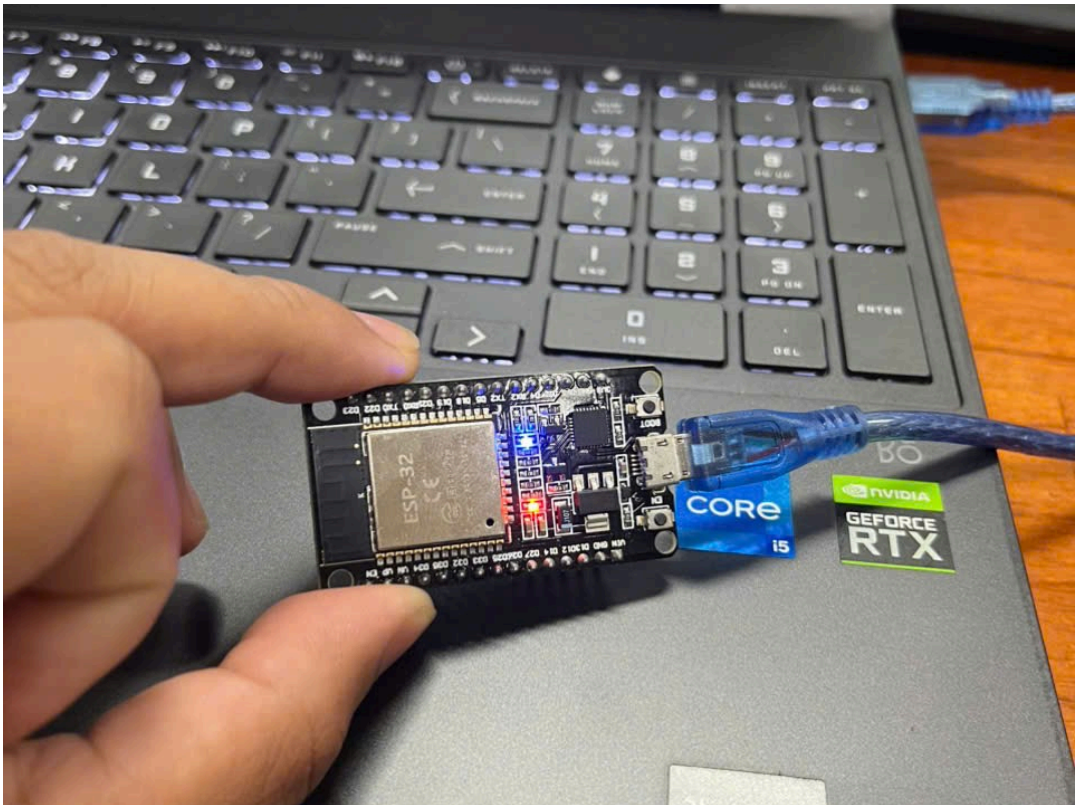
#	Kịch bản	Thao tác trên Wokwi	Kết quả mong đợi
1	Khởi động	Nhấn Start	Serial in WiFi ket noi thanh cong!, MQTT connecting... OK; sau đó cứ 5 giây in Sent: {...}
2	Hoạt động bình thường	Đặt DHT22 khoảng -18 °C, không nhấn nút	LED, buzzer, đèn kho tắt; door_open:false, exposure_sec:0; web hiển thị "An toàn"
3	Nhiệt độ vượt ngưỡng	Nâng DHT22 lên trên -15 °C (ví dụ -10 °C)	LED sáng, buzzer kêu; web hiện "Quá nhiệt" và thêm cảnh báo nhiệt độ
4	Mở cửa ngắn	Giữ nút dưới 3 phút, rồi thả	Đèn kho bật, door_open:true, exposure_sec tăng; thả nút thì đèn tắt, exposure_sec về 0; không có cảnh báo

#	Kịch bản	Thao tác trên Wokwi	Kết quả mong đợi
5	Cửa mở quá lâu	Giữ nút trên 180 giây (có thể đổi ngưỡng thành 30000 ms để demo)	Serial in !!! CẢNH BÁO: Cửa mở quá lâu; publish {"alert":"cua_mo_qua_lau",...} đúng một lần; LED + buzzer bật; web thêm cảnh báo cửa
6	Mất kết nối MQTT	Sai thông tin broker hoặc mất mạng	Serial in MQTT connecting... fail rc=... và thử lại mỗi 5 giây

Mẹo khi kiểm thử: do delay(5000), nút chỉ được đọc mỗi 5 giây, nên cần giữ nút qua ít nhất một mốc lấy mẫu mới được ghi nhận.

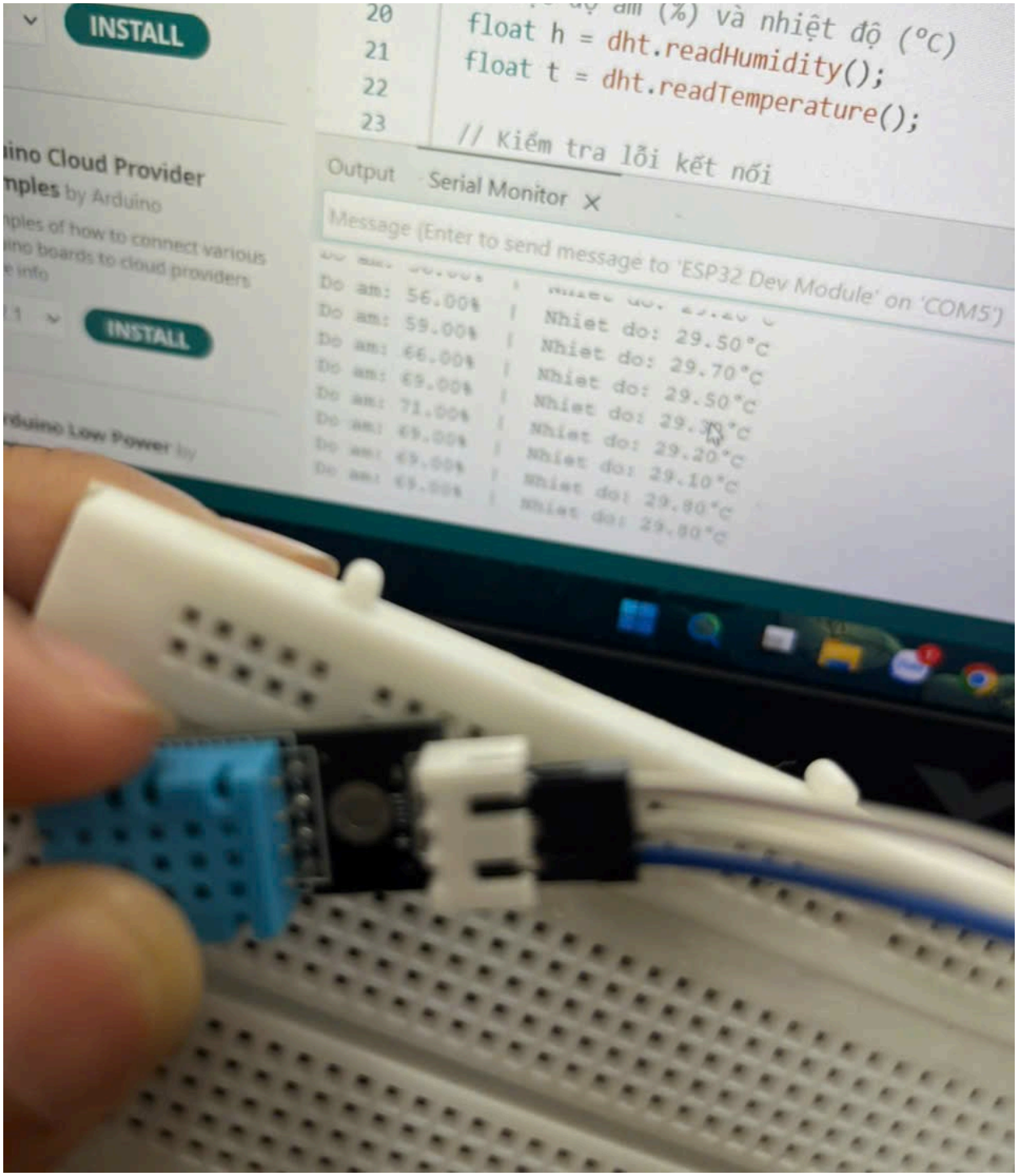
ẢNH THỰC TẾ BÀI LAB

1. Điều khiển ánh sáng (Nháy Led, cảm biến ánh sáng)



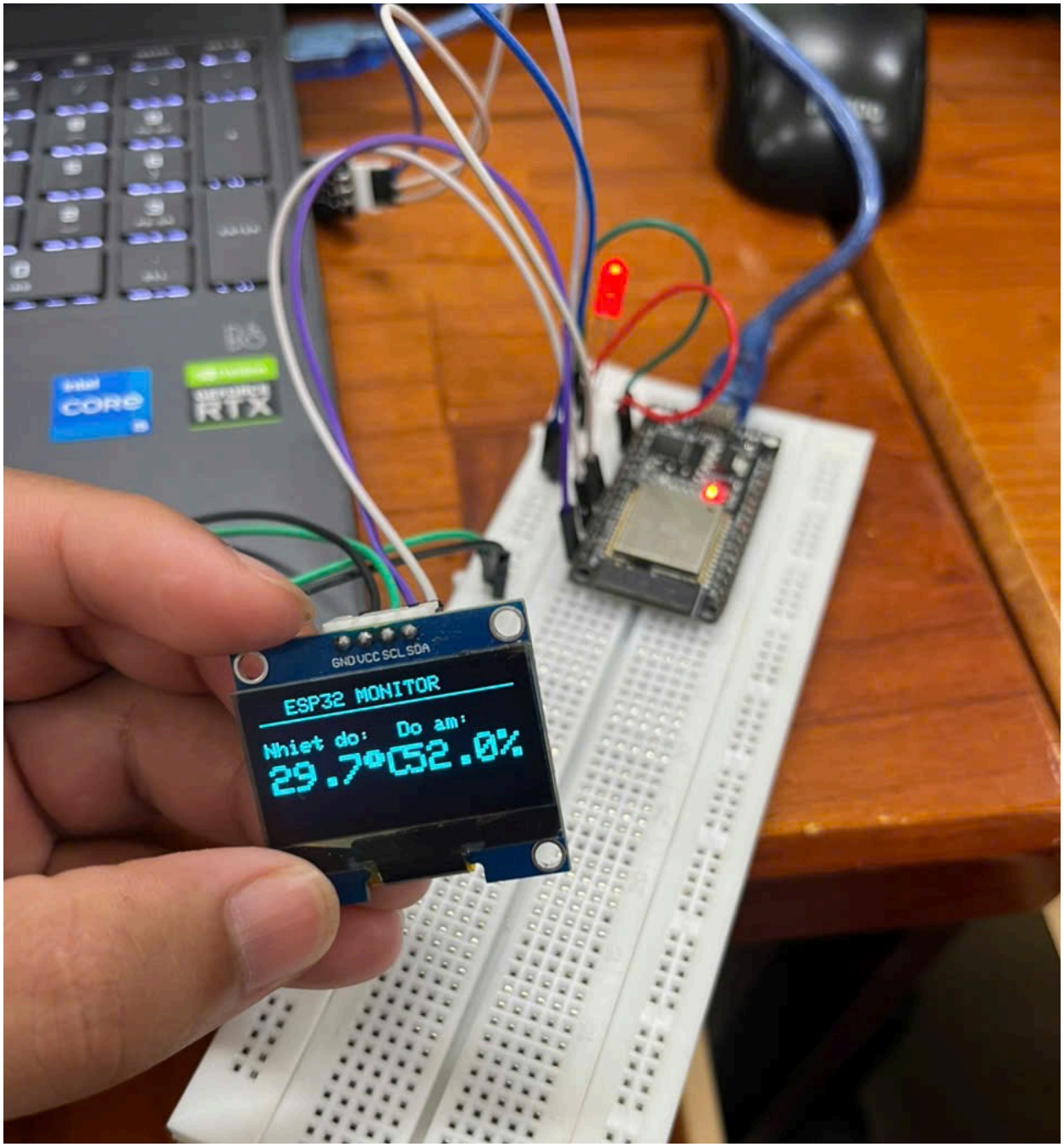
Hình 8.1: Ảnh thực tế điều khiển ánh sáng

2. Đọc Nhiệt độ và độ ẩm từ DHT11/ DHT22 ra log



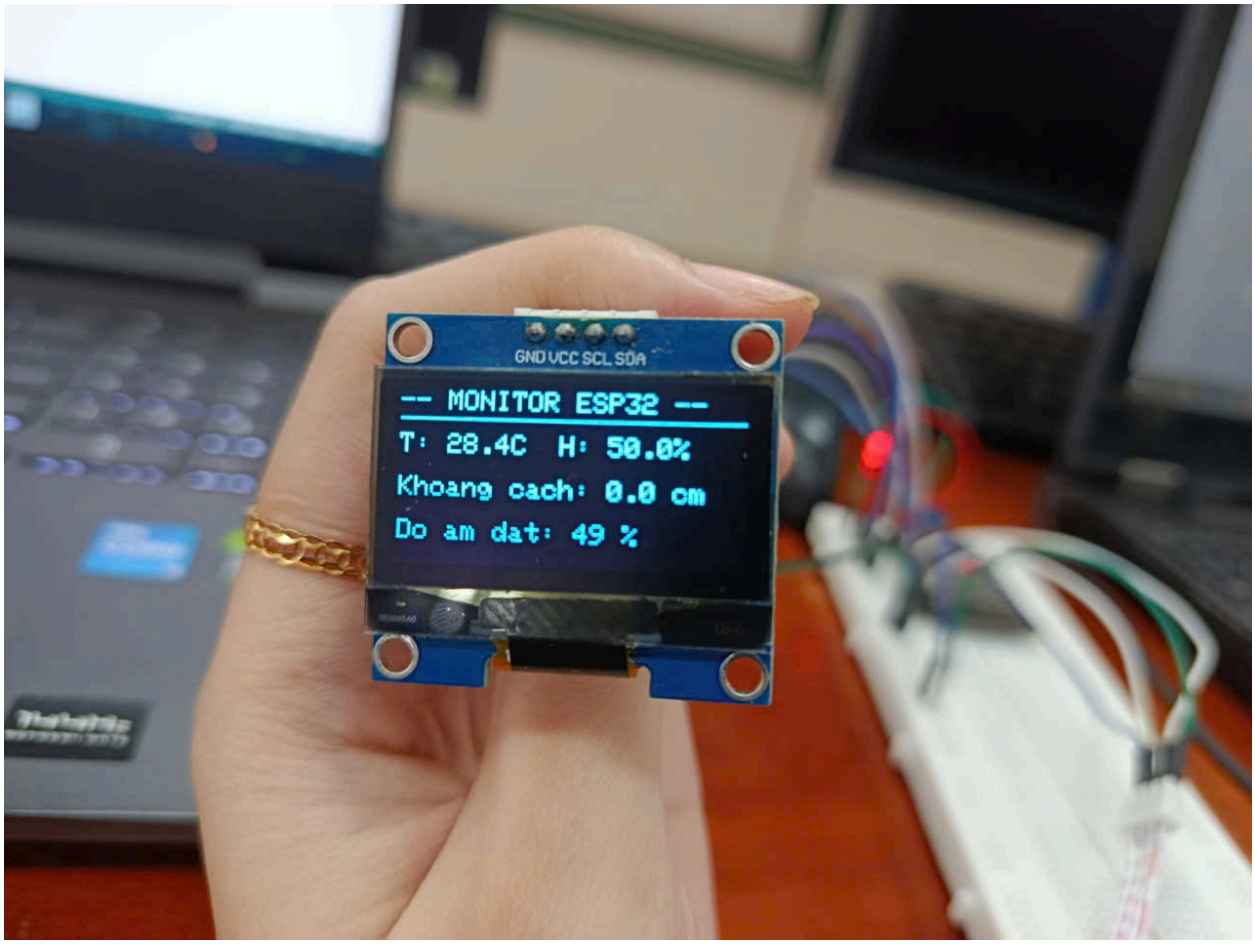
Hình 8.2: Ảnh thực tế đọc nhiệt độ độ ẩm

3. Hiển thị nhiệt độ và độ ẩm từ DHT11 ra màn hình OLED



Hình 8.3: Ảnh thực tế hiển thị nhiệt độ và độ ẩm ra màn hình OLED

4. Đọc và hiển thị nhiệt độ, độ ẩm, khoảng cách (Cảm biến siêu âm), độ ẩm đất

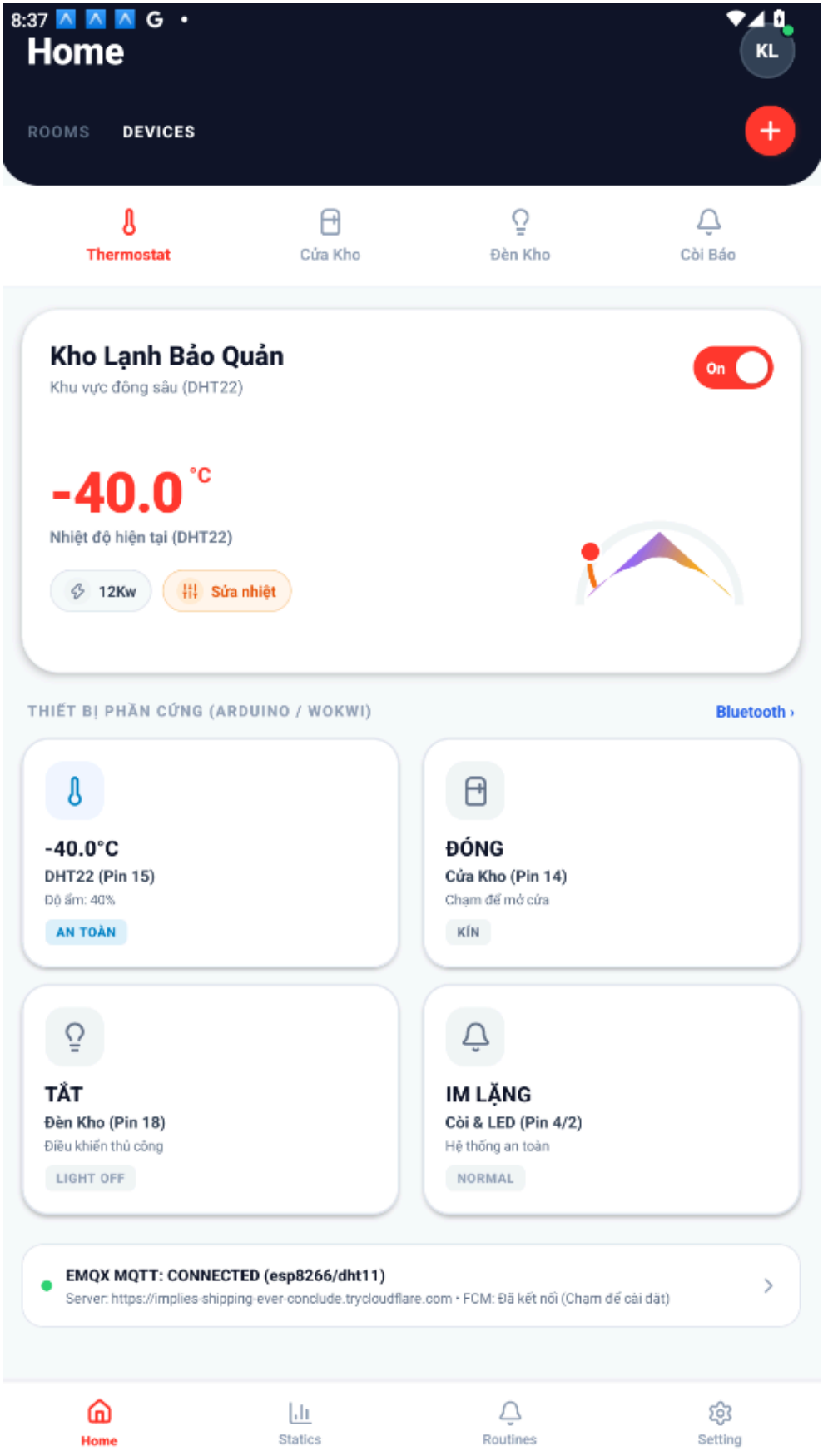


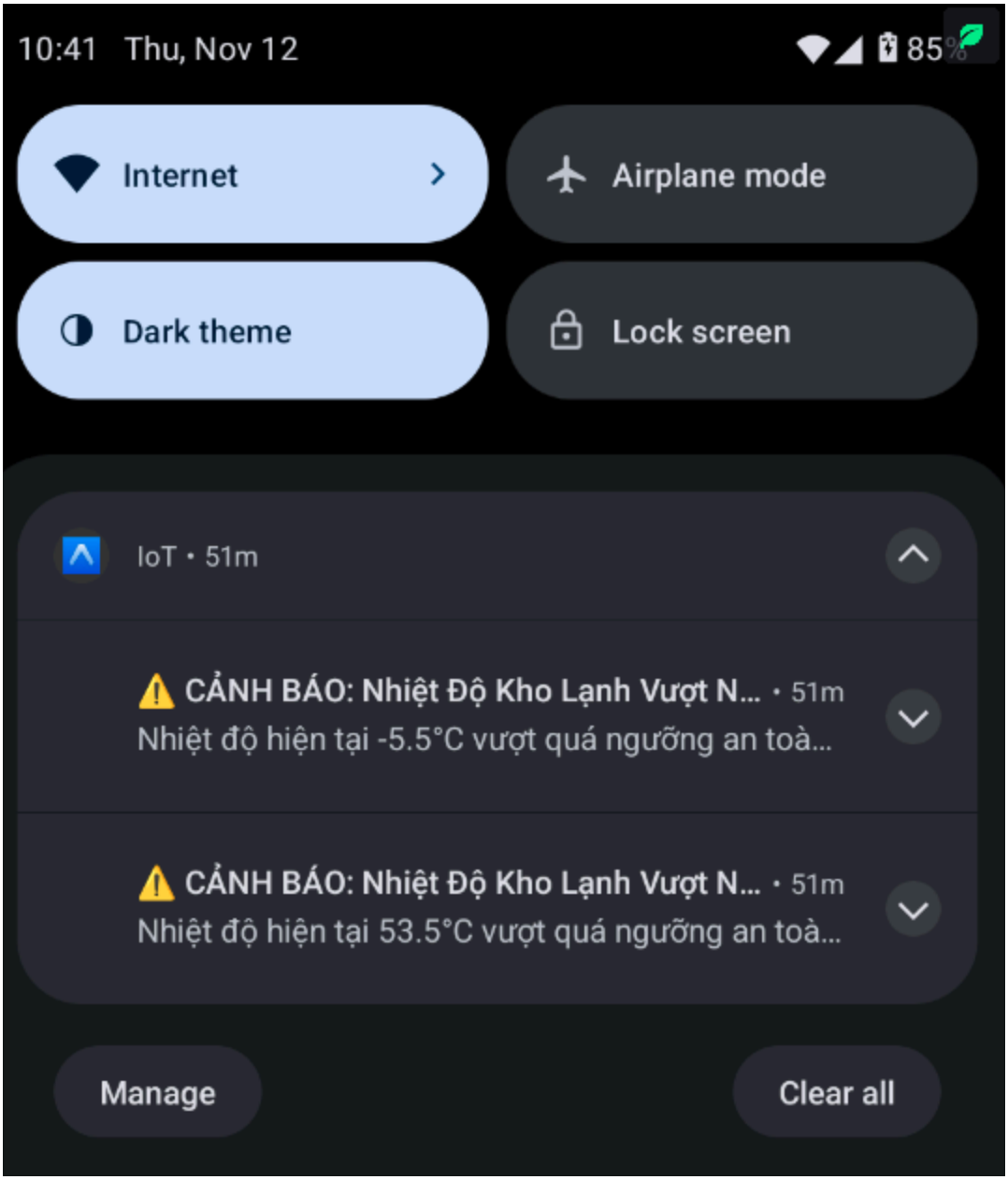
Hình 8.4: Ảnh thực tế đọc và hiển thị nhiệt độ, độ ẩm, khoảng cách (Cảm biến siêu âm), độ ẩm đất

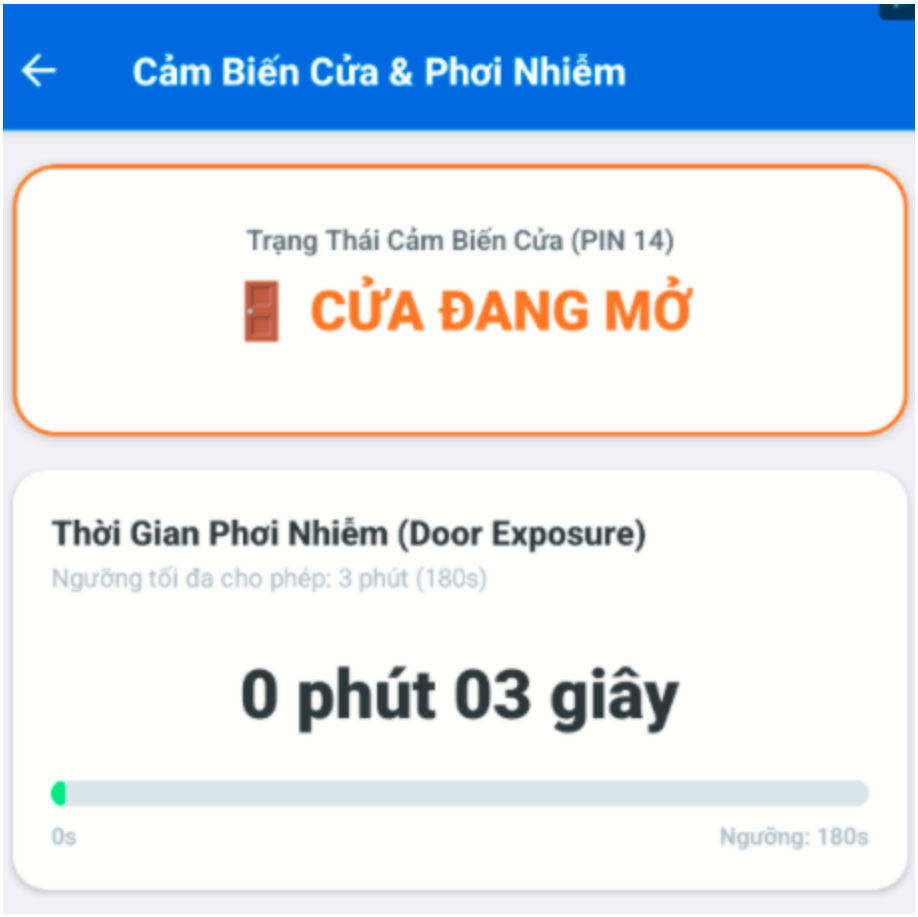
CHƯƠNG 7: Kết luận

Đồ án đã xây dựng được hệ thống giám sát kho lạnh hoàn chỉnh trên Wokwi: ESP32 đo nhiệt độ, độ ẩm, ánh sáng, theo dõi cửa, cảnh báo tại chỗ bằng LED và buzzer, đồng thời đẩy dữ liệu JSON qua MQTT/TLS lên EMQX để dashboard web hiển thị theo thời gian thực. Hai quy tắc cảnh báo cốt lõi (nhiệt độ > -15 °C, cửa mở > 180 giây) hoạt động đúng mô tả. Với các cải tiến về bảo mật, cấu trúc topic và xử lý không chặn, hệ thống có thể chuyển sang phần cứng thật mà giữ nguyên kiến trúc hiện tại.

Giao diện phần mềm trên Expo Go:


8:37
Home
KL
ROOMS
DEVICES
Thermostat
Cửa Kho
Đèn Kho
Còi Báo
Kho Lạnh Bảo Quản
Khu vực đông sâu (DHT22)
On
-40.0 °C
Nhiệt độ hiện tại (DHT22)
12Kw
Sửa nhiệt
THIẾT BỊ PHẦN CỨNG (ARDUINO / WOKWI)
Bluetooth ›
-40.0°C
DHT22 (Pin 15)
Độ ẩm: 40%
AN TOÀN
ĐÓNG
Cửa Kho (Pin 14)
Chạm để mở cửa
KÍN
TẮT
Đèn Kho (Pin 18)
Điều khiển thủ công
LIGHT OFF
IM LẶNG
Còi & LED (Pin 4/2)
Hệ thống an toàn
NORMAL
EMQX MQTT: CONNECTED (esp8266/dht11)
Server: https://implies-shipping-ever-conclude.trycloudflare.com • FCM: Đã kết nối (Chạm để cài đặt)
Home
Statics
Routines
Setting


10:41 Thu, Nov 12
85%
Internet
Airplane mode
Dark theme
Lock screen
IoT • 51m
⚠️ CẢNH BÁO: Nhiệt Độ Kho Lạnh Vượt N... • 51m
Nhiệt độ hiện tại -5.5°C vượt quá ngưỡng an toà...
⚠️ CẢNH BÁO: Nhiệt Độ Kho Lạnh Vượt N... • 51m
Nhiệt độ hiện tại 53.5°C vượt quá ngưỡng an toà...
Manage
Clear all


Cảm Biến Cửa & Phơi Nhiễm
Trạng Thái Cảm Biến Cửa (PIN 14)
CỬA ĐANG MỞ
Thời Gian Phơi Nhiễm (Door Exposure)
Ngưỡng tối đa cho phép: 3 phút (180s)
0 phút 03 giây
0s
Ngưỡng: 180s

Thống Kê Min / Max & Nhật Ký Đo (50 bản ghi)

Thấp nhất	Trung bình	Cao nhất
-21.3°C	-21.3°C	-21.3°C

10:45:42	-21.3°C	40.0%	Đóng
10:45:37	-21.3°C	40.0%	Cửa Mở
10:38:07	-21.3°C	40.0%	Đóng
09:56:19	-21.3°C	40.0%	Đóng
09:56:12	-21.3°C	40.0%	Đóng
09:56:07	-21.3°C	40.0%	Đóng
09:56:01	-21.3°C	40.0%	Đóng
09:55:55	-21.3°C	40.0%	Đóng
09:55:50	-21.3°C	40.0%	Đóng
09:55:45	-21.3°C	40.0%	Đóng

Xóa lịch sử đo